VICTORIA PARK ACADEMY SHENZHEN

CIE IGCSE COMPUTER SCIENCE (0478) — YEAR 9 SEMESTER 2

WEEKS 7-12: ITERATION, ARRAYS, STRINGS, FUNCTIONS & MODULAR PROGRAMMING

Document: 02_VPA_CS_Y9S2_Weeks_7-12.txt

DOCUMENT OVERVIEW

This document contains complete lesson plans, worksheets, and answer keys for Weeks 7 through 12 of Year 9 Semester 2, covering:

- Week 7: Iteration — FOR & WHILE Loops
- Week 8: 1D Arrays / Lists
- Week 9: 2D Arrays / Nested Lists
- Week 10: String Handling
- Week 11: Functions
- Week 12: Scope & Modular Programming

Format: 12 lessons, 12 worksheets (XA + XB per week), 6 answer keys.

All lessons are 40 minutes. Programming language: Python (Thonny IDE).

Students: EAL, mixed ability. Bilingual support: English + Chinese + pinyin.

Prerequisites from Weeks 1-6: Algorithms, pseudocode, flowcharts, searching, Python basics (variables, I/O, operators, selection/if statements).

WEEK 7: ITERATION — FOR & WHILE LOOPS

LESSON 7.1 — FOR Loops: Counting & Iterating

Topic: FOR loops — counting loops, range(), iterating through lists

Duration: 40 minutes

Lesson Type: Programming / Conceptual

Syllabus Ref: 2.2.2 (Sequence, Selection, Iteration)

Learning Objective:

By the end of this lesson, students will be able to write FOR loops in Python using range() to repeat code a set number of times and iterate through lists.

Key Vocabulary:

- loop / 循环 (xún huán) — repeating a block of code

- FOR loop / 计数循环 (jì shù xún huán) — count-controlled loop

- range() / 范围函数 (fàn wéi hán shù) — generates a sequence of numbers

- iterate / 迭代 (dié dài) — go through each item one by one

- index / 索引 (suǒ yǐn) — position number in a list

- list / 列表 (liè biǎo) — ordered collection of items

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| Display on board:
| "Without using a loop, write Python code to print the 7 times
| table from 7 x 1 to 7 x 12."
| Students write in notebooks. After 2 minutes, ask:
| "Who found that boring? Who made a typo?"
| Introduce: "There is a better way — loops!"

3-12 min | DIRECT INSTRUCTION

| Write title: "FOR Loops — The Python Way to Repeat"
| CONCEPT 1: What is a loop?
| A loop repeats a block of code without you copying it.
| Pseudocode: FOR i ← 1 TO 10
| OUTPUT i
| ENDFOR
| CONCEPT 2: range() in Python
| range(5) → 0, 1, 2, 3, 4
| range(1, 6) → 1, 2, 3, 4, 5
| range(0, 10, 2) → 0, 2, 4, 6, 8
| Write these on board with arrows. Ask students to predict outputs.
| CONCEPT 3: Basic FOR loop syntax

```
for i in range(5):
    print(i)
```

| CRITICAL: Indentation! Python uses spaces (usually 4).
| Show error: what happens without indentation?
| CONCEPT 4: FOR loop with lists

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

| CULTURAL EXAMPLE:

```
cities = ["Shenzhen", "Beijing", "Shanghai", "Guangzhou"]
for city in cities:
    print("I want to visit", city)
```

12-25 min | GUIDED PRACTICE

| Task 1 (whole class, teacher codes live):
| Write a FOR loop that prints the 7 times table.

```
for i in range(1, 13):
```

| print(f"7 x {i} = {7 * i}")

| Task 2 (pairs, mini-whiteboards):

| "Write a FOR loop to print all even numbers from 2 to 20."

| Answer: for i in range(2, 21, 2): print(i)

| Task 3 (teacher live-codes):

| Count vowels in a string.

| word = "Shenzhen"

| count = 0

| for letter in word:

| if letter in "aeiouAEIOU":

| count = count + 1

| print(f"There are {count} vowels")

| Compare: "This 8-line loop replaces how many IF statements?"

| Answer: Would need 8 separate IFs for an 8-letter word!

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 7A.

| Students complete coding tasks at computers (Thonny).

| Tasks include: print times tables, sum numbers 1-100,

| count consonants in a word, print a word backwards.

| Teacher circulates, checks screens, helps with indentation errors.

35-40 min | PLENARY + HOMEWORK

| Plenary: "Thumbs up/middle/down — how confident are you with FOR loops?"

| Ask: "When is a FOR loop better than writing the same code many times?"

| Expected: "When you know HOW MANY times to repeat."

| Homework (WORKSHEET 7B):

| "Complete trace table problems and write two FOR loop programs."

| Challenge: "Write a FOR loop that counts how many letters 'n' are in

| the word 'Internationalization'."

EAL Support:

- Display loop structure template on board with colour-coded keywords
- Provide printed bilingual vocabulary card on each desk
- Pair EAL students with bilingual buddies for guided practice
- Use visual: draw arrow showing flow of loop iterations
- Sentence starters: "The loop runs ___ times" / "循环运行 ___ 次"

Differentiation:

Emerging:

- Use worksheet with pre-filled code (fill in the blanks only)
- Pair with Secure student
- Allow use of printed range() reference card
- Focus on range(5) and simple list iteration only

Developing:

- Standard worksheet tasks
- Provide hints (commented code skeletons)
- Extend to range(start, stop) patterns

Secure:

- Extension: nested pattern problems (print triangle of stars)
- Write loops with step values
- Peer tutor Emerging students
- Challenge: "Print all prime numbers from 1 to 50 using a FOR loop"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- Printed WORKSHEET 7A + 7B

- Bilingual vocabulary cards (loop terms)

- range() quick reference card

LESSON 7.2 — WHILE Loops & Number Guessing Game

Topic: WHILE loops — condition-controlled, sentinel values, infinite loops

Duration: 40 minutes

Lesson Type: Programming / Practical

Syllabus Ref: 2.2.2 (Iteration)

Learning Objective:

By the end of this lesson, students will be able to write WHILE loops with conditions, understand sentinel values, avoid infinite loops, and choose between FOR and WHILE for different problems.

Key Vocabulary:

- WHILE loop / 条件循环 (tiáo jiàn xún huán) — condition-controlled loop

- condition / 条件 (tiáo jiàn) — test that decides if loop continues

- infinite loop / 无限循环 (wú xiàn xún huán) — loop that never stops

- sentinel value / 哨兵值 (shào bīng zhí) — special value that ends input

- break / 中断 (zhōng duàn) — exit a loop early

- Boolean / 布尔值 (bù ěr zhí) — True or False value

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| "Look at this code. What will it print? Will it ever stop?"

| x = 1

| while x < 5:

| print(x)

| Students write prediction. Reveal: INFINITE LOOP (x never changes).

| Discuss: "How do we fix it?" → Add x = x + 1 inside the loop.

3-12 min | DIRECT INSTRUCTION

| Write title: "WHILE Loops — Keep Going While True"

| CONCEPT 1: WHILE loop syntax

| while condition:

| # code to repeat

| Pseudocode: WHILE condition DO

| statements

| ENDWHILE

| CONCEPT 2: How it works

| 1. Check condition → if True, run code

| 2. Check condition again → if True, run code again

| 3. Repeat until condition is False → exit loop

| Show flowchart on board: diamond (condition) → rectangle (code)

| → arrow back to diamond.

| CONCEPT 3: FOR vs WHILE — When to use which?

| ┌──────────────────────┬──────────────────────────────┐

| FOR loop | WHILE loop |

| You know the count | You DON'T know the count |

| "Repeat 10 times" | "Repeat until user says stop"|

| Range is fixed | Condition controls exit |

| Less error-prone | More flexible |

| CONCEPT 4: Infinite loops — THE DANGER

| If the condition never becomes False, the loop runs FOREVER.

| Demo: Show infinite loop in Thonny. Press STOP button (red square).

| ALWAYS ensure something inside the loop can change the condition.

| CONCEPT 5: Sentinel value

| A special input value that tells the program to stop.

| Example: Enter scores until user types -1.

| score = int(input("Enter score (-1 to stop): "))

| while score != -1:

```
|     print(f"Score: {score}")
|     score = int(input("Enter score (-1 to stop): "))
```

12-25 min | GUIDED PRACTICE

```
| Task 1 (teacher codes live):
| Count-up program: count from 1 to 10 using WHILE.
|   count = 1
|   while count <= 10:
|       print(count)
|       count = count + 1
| Task 2 (pairs, mini-whiteboards):
| "Write a WHILE loop that counts DOWN from 10 to 1 (like a rocket launch!)."
| Answer: count = 10; while count > 0: print(count); count = count - 1
| Task 3 (teacher codes live — NUMBER GUESSING GAME):
|   import random
|   secret = random.randint(1, 100)
|   guess = 0
|   while guess != secret:
|       guess = int(input("Guess the number (1-100): "))
|       if guess < secret:
|           print("Too low!")
|       elif guess > secret:
|           print("Too high!")
|   print("Correct! Well done!")
| Discuss: "Why is WHILE better than FOR here?"
| Answer: "We don't know how many guesses the user needs."
```

25-35 min | INDEPENDENT PRACTICE

```
| Distribute WORKSHEET 7B (in-class portion).
| Students choose ONE of:
|   A) Complete the number guessing game (add attempt counter)
|   B) Create a simple menu system:
|       1. Say Hello  2. Show date  3. Exit
|       (loops until user chooses 3)
|   C) Extension: Modify guessing game to give max 7 attempts
| Teacher circulates, helps with infinite loop bugs.
```

35-40 min | PLENARY + HOMEWORK

```
| Plenary: "Stand up if... you know when to use FOR vs WHILE"
```

| "Sit down if... you know what an infinite loop is"

| "Clap if... you completed the guessing game!"

| Key question: "Your friend writes a WHILE loop that never stops.

| What three things should you check?"

| Answer: (1) Is the condition correct?

| (2) Does something inside change the variable?

| (3) Will the condition eventually become False?

| Homework: Complete remaining WORKSHEET 7B trace table problems.

| Write in notebook: "Explain in 3 sentences when to use FOR and

| when to use WHILE." (English or Chinese acceptable)

EAL Support:

- WHILE vs FOR comparison table on wall poster (bilingual)

- Use "traffic light" analogy: WHILE = green light keeps you going

- Visual flowchart for WHILE loop on board

- Sentence frames: "Use FOR when you know ___" / "Use WHILE when ___"

- Pre-teach "sentinel" — explain as a guard/soldier who says "stop!"

Differentiation:

Emerging:

- Provide complete code with comments; student fills in conditions only
- Focus on simple WHILE loop (count from 1 to 5)
- Use trace table worksheet to trace execution step by step
- Pair with Developing student for guessing game

Developing:

- Standard worksheet — complete menu system task
- Add attempt counter to guessing game
- Complete comparison table: FOR vs WHILE

Secure:

- Extension: Add difficulty levels to guessing game (easy: 1-50, hard: 1-500)
- Create a login system: 3 attempts max, lock after failures
- Peer tutor — explain difference to Emerging students
- Challenge: Implement binary search guessing (computer guesses!)

Resources Needed:

- Thonny IDE

- Projector for live coding

- Mini-whiteboards

- WORKSHEET 7B

- STOP sign card for infinite loop discussion (red card)

- FOR vs WHILE comparison poster

WORKSHEET 7A — FOR Loops Practice

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Vocabulary Check (词汇检查)

Fill in the blanks with the correct term.

1. A _____ repeats a block of code multiple times. (循环)
2. The Python function _____ generates a sequence of numbers. (范围函数)
3. Going through each item in a list one by one is called _____. (迭代)
4. A FOR loop is a _____-controlled loop — you know how many times it runs. (计数)
5. The position number of an item in a list is called its _____. (索引)

Section B: Predict the Output (预测输出)

Write what each program will print.

6. for i in range(4):
   print(i)

   Output: _____

7. for i in range(1, 6):
   print(i * 2)

   Output: _____

8. for i in range(10, 0, -2):
   print(i)

   Output: _____

9. word = "PYTHON"
   for letter in word:
   print(letter)

   Output: _____

Section C: Trace Table (跟踪表)

Trace this FOR loop. Complete the table.

```
total = 0
for i in range(1, 6):
    total = total + i
    print(f"i={i}, total={total}")
```

| Iteration (迭代) | i | total |

Final value of total: _____

Section D: Write the Code (写代码)

Write Python FOR loops for each task. Test in Thonny.

10. Print the 9 times table from 9 x 1 to 9 x 12.

10. Create a list of 5 Chinese cities. Print each city with "I love ".
11. Count how many times the letter 'a' appears in "banana".

13. EXTENSION: Print this pattern using a FOR loop:

```
*
**
***
```

WORKSHEET 7B — WHILE Loops & Loop Comparison

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: FOR vs WHILE (选择正确的循环)

For each problem, tick (✓) whether FOR or WHILE is better.

Problem	FOR	WHILE
1. Print "Hello" 10 times		
2. Keep asking for password until correct		
3. Add all numbers from 1 to 100		
4. Menu system until user chooses Exit		
5. Check each item in a shopping list		

Section B: Trace Table for WHILE Loop (跟踪 WHILE 循环)

Trace this program. Complete the table.

```
x = 5
total = 0
while x > 0:
    total = total + x
    x = x - 1
    print(f"x={x}, total={total}")
```

(Note: x is updated BEFORE printing in this version)

Step	x (before)	x > 0?	total (after)	x (after)
1	5			

Final total: _____ Final x: _____

Section C: Find the Bug (找错误)

Each program has an error. Identify and fix it.

6. This should print 1, 2, 3, 4, 5 but has a problem:

```
count = 1
while count < 5:
    print(count)
    count = count + 1
```

Bug: _____

Fix: _____

7. This causes an infinite loop:

```
x = 10
while x > 0:
    print(x)
```

Bug: _____

Fix: _____

8. This never prints anything:

```
x = 0
while x > 5:
    print(x)
    x = x + 1
```

Bug: _____

Fix: _____

Section D: Coding Challenge (编程挑战)

Write in Thonny. Save your files.

9. MENU SYSTEM: Write a program that displays a menu:

12. Print Hello
13. Print Goodbye
14. Exit

Keep showing the menu until the user enters 3.

Use a WHILE loop. Test it works!

[Write your code here or on computer]

15. EXTENSION: Write a program that keeps asking the user to enter a

number. Add each number to a running total. When the user enters 0, stop and print the total. Use a WHILE loop with a sentinel value.

[Write your code here or on computer]

ANSWER KEY — WEEK 7 WORKSHEETS

WORKSHEET 7A ANSWERS:

Section A:

16. loop / 循环
17. range()
18. iterating / 迭代
19. count / 计数
20. index / 索引

Section B:

21. 0 1 2 3 (each on a new line)
22. 2 4 6 8 10 (each on a new line)
23. 10 8 6 4 2 (each on a new line)

9. P Y T H O N (each on a new line)

Section C:

Iteration	i	total
2	2	3
3	3	6
4	4	10
5	5	15

Final value of total: 15

Section D:

24.
```
for i in range(1, 13):
    print(f"9 x {i} = {9 * i}")
```
25.
```
cities = ["Shenzhen", "Beijing", "Shanghai", "Guangzhou", "Hangzhou"]
for city in cities:
    print("I love " + city)
```
(Other city lists accepted)

26.
```
word = "banana"
count = 0
for letter in word:
    if letter == "a":
        count = count + 1
print(f"'a' appears {count} times")
```

Output: 3

27. for i in range(1, 6):
    print("*" * i)

(Alternative: nested loop accepted)

WORKSHEET 7B ANSWERS:

Section A:

Problem	FOR	WHILE
1. Print "Hello" 10 times	✓	
2. Keep asking for password until correct		✓
3. Add all numbers from 1 to 100	✓	
4. Menu system until user chooses Exit		✓
5. Check each item in a shopping list	✓	

Section B:

Step	x (before)	x > 0?	total (after)	x (after)
1	5	True	5	4
2	4	True	9	3
3	3	True	12	2
4	2	True	14	1
5	1	True	15	0

(Then x=0, condition False, loop exits)

Final total: 15 Final x: 0

Section C:

28. Bug: Condition is count < 5, so it only prints 1,2,3,4 (stops before 5)
    Fix: Change to while count <= 5: OR change to while count < 6:

29. Bug: x never changes inside the loop (no x = x - 1 or similar)
    Fix: Add x = x - 1 inside the loop body

8. Bug: x starts at 0, condition x > 5 is False immediately

Fix: Change starting value to x = 1, or change condition

Section D:

9. Sample answer:

choice = 0

```
while choice != 3:
    print("1. Print Hello")
    print("2. Print Goodbye")
    print("3. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        print("Hello")
    elif choice == 2:
        print("Goodbye")
print("Goodbye!")
```

10. Sample answer:

```
total = 0
number = int(input("Enter a number (0 to stop): "))
while number != 0:
    total = total + number
    number = int(input("Enter a number (0 to stop): "))
print(f"Total is: {total}")
```

CIE-Style Exam Question (Bonus, 4 marks):

"A program needs to read student marks until -1 is entered. Each mark should be added to a total. Explain why a WHILE loop is more suitable than a FOR loop for this problem."

Mark Scheme:

- FOR loop requires knowing the number of marks in advance [1]
- WHILE loop can repeat until the sentinel value (-1) is entered [1]
- The number of student marks is unknown at the start [1]
- WHILE checks a condition (mark != -1) before continuing [1]

Sample Answer:

> The number of student marks is not known before the program runs. A FOR loop requires a fixed number of iterations. A WHILE loop can continue until the sentinel value (-1) is entered, making it more suitable for input of unknown length.

WEEK 8: 1D ARRAYS / LISTS

LESSON 8.1 — Lists in Python: Creating, Accessing, Modifying

Topic: Lists — creating, indexing, accessing, list operations (append, insert,

remove, pop, len())

Duration: 40 minutes

Lesson Type: Programming / Conceptual

Syllabus Ref: 2.2.3 (1D Arrays)

Learning Objective:

By the end of this lesson, students will be able to create Python lists, access elements by index, and use common list operations to add, remove, and modify data.

Key Vocabulary:

- list / 列表 (liè biǎo) — ordered collection of items
- element / 元素 (yuán sù) — a single item in a list
- index / 索引 (suǒ yǐn) — position number (starts at 0 in Python)
- append / 追加 (zhuī jiā) — add to the end of a list
- insert / 插入 (chā rù) — add at a specific position
- remove / 移除 (yí chú) — delete an element by value
- pop / 弹出 (tán chū) — remove and return the last element
- length / 长度 (cháng dù) — number of items (len())

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| "Without lists, declare 5 variables to store student names:
| student1 = "Alice"
| student2 = "Bob"
| What is wrong with this approach?"
| Students discuss in pairs. Expected: "Too many variables,
| hard to change, can't use a loop."
| Introduce: "Lists solve all these problems!"

3-12 min | DIRECT INSTRUCTION

| Write title: "Lists — Your First Data Structure"
| CONCEPT 1: What is a list?
| A list stores multiple items in ONE variable.
| students = ["Alice", "Bob", "Charlie", "Diana"]
| marks = [85, 92, 78, 90]
| mixed = ["Hello", 42, 3.14, True] # Python allows mixed types!
| CONCEPT 2: Indexing — ZERO-BASED!
| Python starts counting at 0, NOT 1.
| students = ["Alice", "Bob", "Charlie", "Diana"]

| Index: 0 1 2 3

| print(students[0]) → "Alice"

| print(students[2]) → "Charlie"

| print(students[-1]) → "Diana" (negative = from end!)

| COMMON ERROR: students[4] → IndexError! There is no index 4.

| The last valid index is len(list) - 1.

| CONCEPT 3: Modifying lists

| Change an element: students[1] = "Ben"

| Add to end: students.append("Eve")

| Insert at index: students.insert(1, "Amy")

| Remove by value: students.remove("Charlie")

| Remove by index: students.pop(2) (also returns the value)

| Get length: len(students)

| CONCEPT 4: CULTURAL EXAMPLE — Class Register

| register = ["Li Wei", "Wang Fang", "Zhang Ming",

| "Liu Hua", "Chen Jie"]

| print(f"Class has {len(register)} students")

| register.append("Huang Xin")

| print(f"Now class has {len(register)} students")

12-25 min | GUIDED PRACTICE

| Task 1 (teacher codes live):

| Create a list of 5 subjects, print each with index.

| subjects = ["Maths", "English", "Science", "CS", "Chinese"]

| print(subjects[0])

| print(subjects[1])

| Ask: "How can we do this with a FOR loop?"

| for i in range(len(subjects)):

| print(f"{i}: {subjects[i]}")

| Task 2 (pairs, mini-whiteboards):

| "Given: scores = [78, 85, 92, 67, 88]

| Write code to change the lowest score to 70."

| Answer: scores[3] = 70

| Task 3 (teacher codes live):

| Build a class register program.

| register = []

| name = input("Enter student name (or 'stop'): ")

| while name != "stop":

| register.append(name)

| name = input("Enter student name (or 'stop'): ")

| print(f"Register: {register}")

| print(f"Total students: {len(register)}")

| Connect to Week 7: "We use a WHILE loop because we don't know

| how many students the user will enter!"

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 8A.

| Students work on computers (Thonny) to:

| - Create lists and access elements

| - Build a simple class register

| - Add/remove students

| - Handle index errors

| Teacher circulates, checks for IndexError understanding.

35-40 min | PLENARY + HOMEWORK

| Plenary: Quick-fire questions

| Q1: "What index is the FIRST element?" → 0

| Q2: "What function adds to the end?" → append()

| Q3: "What happens if I access index 5 in a 3-item list?" → Error

| Q4: "How do I get the number of items?" → len()

| Homework: Complete WORKSHEET 8A Section D coding tasks.

| "In your notebook: Explain why Python uses 0-based indexing.

| Write 3 sentences." (Research question)

EAL Support:

- Colour-coded index diagram on board: 0=red, 1=blue, 2=green, etc.

- Physical: use student line-up — "Who is at index 0? Index 2?"

- Bilingual list operation reference card on each desk

- Visual diagram showing memory with labelled boxes for each index

- Sentence starters: "The element at index ___ is ___"

Differentiation:

Emerging:

- Pre-filled code for register program (fill blanks only)
- Work with provided lists (don't need to create from input)
- Focus on basic indexing and append() only
- Use worksheet with visual index boxes drawn

Developing:

- Standard worksheet — create register, add/remove students
- Introduce insert() and remove()
- Handle simple errors
- Build a to-do list program

Secure:

- Extension: Build a quiz program using lists for questions and answers
- Implement search within a list ("Is 'Alice' in the register?")
- Use list methods not yet taught (sort(), reverse())
- Peer tutor Emerging students
- Challenge: "Write a program that finds the second highest mark in a list"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- Printed WORKSHEET 8A + 8B

- Bilingual list operation reference cards

- Index visual diagram (laminated)

LESSON 8.2 — Traversing Lists, Finding Max/Min/Sum, Linear Search

Topic: Traversing lists with FOR loops, calculating statistics, linear search

Duration: 40 minutes

Lesson Type: Programming / Practical

Syllabus Ref: 2.2.3 (1D Arrays), 2.2.1 (Searching)

Learning Objective:

By the end of this lesson, students will be able to traverse a list using a

FOR loop, calculate max/min/sum/average, and implement linear search in Python.

Key Vocabulary:

- traverse / 遍历 (biàn lì) — go through every element

- maximum / 最大值 (zuì dà zhí) — largest value

- minimum / 最小值 (zuì xiǎo zhí) — smallest value

- average / 平均值 (píng jūn zhí) — sum divided by count

- linear search / 线性搜索 (xiàn xìng sōu suǒ) — check each item in order

- accumulator / 累加器 (lěi jiā qì) — variable that builds up a total

Lesson Structure (40 Minutes):

0-3 min | **DO NOW**

| "Look at this list: marks = [78, 92, 65, 88, 91]

| Without using Python's built-in functions, how would you find

| the highest mark? Write your idea in 2 sentences."
| Collect ideas. Expected: "Check each number, keep track of
| the biggest one seen so far."

3-12 min | DIRECT INSTRUCTION

| Write title: "Processing Lists — Real Programming Power"
| CONCEPT 1: Traversing with FOR
| "To do something with every item, use a FOR loop."

```
|   marks = [78, 92, 65, 88, 91]
|   for mark in marks:
|       print(mark)
```

| Or with index:

```
|   for i in range(len(marks)):
|       print(f"Student {i+1}: {marks[i]}")
```

| CONCEPT 2: Finding the SUM (accumulator pattern)

```
|   total = 0                ← Start with nothing
|   for mark in marks:
|       total = total + mark   ← Add each mark
|   print(f"Total: {total}")   ← Result after loop
```

| CONCEPT 3: Finding MAXIMUM

```
|   highest = marks[0]        ← Assume first is highest
|   for mark in marks:
|       if mark > highest:
|           highest = mark     ← Found a bigger one!
|   print(f"Highest: {highest}")
```

| CONCEPT 4: Finding MINIMUM (same pattern)

```
|   lowest = marks[0]
|   for mark in marks:
|       if mark < lowest:
|           lowest = mark
```

| CONCEPT 5: AVERAGE

```
|   average = total / len(marks)
|   (Must calculate total first!)
```

| CONCEPT 6: LINEAR SEARCH in Python

```
|   target = int(input("Which mark to find? "))
|   found = False
|   for i in range(len(marks)):
```

| if marks[i] == target:
| print(f"Found at position {i}")
| found = True
| if not found:
| print("Not found")
| "This is the Python version of the linear search algorithm
| we learned in Week 3!"

12-25 min | GUIDED PRACTICE

| Task 1 (teacher codes live — STUDENT MARKS PROGRAM):
| Complete program that inputs marks, finds highest, lowest,
| total, and average.
| Task 2 (pairs, mini-whiteboards):
| "Given: temperatures = [22, 25, 19, 28, 24, 21, 26]
| Write code to find how many days were above 25 degrees."
| Answer:
| count = 0
| for temp in temperatures:
| if temp > 25:
| count = count + 1
| Task 3 (teacher codes):
| Enhance the marks program to also find how many students passed
| (mark >= 50).
| pass_count = 0
| for mark in marks:
| if mark >= 50:
| pass_count = pass_count + 1
| print(f"{pass_count} students passed")

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 8B.
| Students build the Student Marks Program:
| - Input 5 student marks into a list
| - Calculate and display: total, average, highest, lowest
| - Count how many passed (>= 50)
| - Implement linear search for a specific mark
| Extension: Find how many got distinction (>= 85)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Show me" check

| "Raise your hand if your program can..."

| - ...calculate the average [hands up]

| - ...find the highest mark [hands up]

| - ...search for a mark [hands up]

| Key question: "Why do we set highest = marks[0] at the start?

| Why not set highest = 0?"

| Answer: "If all marks are negative (or below 0), highest=0

| would be wrong. Using the first element always works."

| Homework: Complete WORKSHEET 8B coding tasks.

| "Research: Python has built-in functions max(), min(), sum().

| How would you use them? Write an example in your notebook."

EAL Support:

- Accumulator pattern visual: draw a box that gets bigger

- Max/min algorithm: "keep the champion" analogy (tournament style)

- Connect to Week 3 linear search — "we're doing the same thing in code now"

- Trace table template for loop iterations

- Bilingual maths terms: sum=和, average=平均, max=最大, min=最小

Differentiation:

Emerging:

- Provide complete code skeleton; student fills in 3-4 key lines
- Use built-in len() and sum() after manual calculation
- Trace table worksheet to follow execution step by step
- Focus on sum and average only

Developing:

- Standard worksheet — full marks program
- Add pass/fail counting
- Implement linear search
- Use both element-based and index-based FOR loops

Secure:

- Extension: Find the TWO highest marks
- Count marks in each grade band (A: 80+, B: 70-79, C: 60-69...)
- Implement search that returns ALL positions of a value
- Peer tutor Emerging students
- Challenge: "Without using sort(), write code to put the list in order"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- WORKSHEET 8B

- Trace table templates

- Student marks program starter code

WORKSHEET 8A — Lists: Creating and Modifying

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Fill in the Blanks (填空)

30. A _____ stores multiple items in a single variable. (列表)
31. In Python, the first element of a list has index _____. (索引)
32. The function _____ adds an element to the end of a list. (追加)
33. The function _____ returns the number of items in a list. (长度)
34. To remove an element by its value, use the _____ method. (移除)

Section B: Predict the Output (预测输出)

For each code snippet, write the final output.

35. fruits = ["apple", "banana", "cherry"]
    print(fruits[1])
    Output: _____
36. numbers = [10, 20, 30, 40]
    numbers[2] = 99
    print(numbers)
    Output: _____
37. colours = ["red", "green"]
    colours.append("blue")
    colours.insert(1, "yellow")
    print(colours)
    Output: _____
38. items = ["pen", "book", "ruler"]
    items.pop()
    print(items)
    print(len(items))
    Output: _____

Section C: What is the Index? (索引是什么?)

For the list: cities = ["Shenzhen", "Beijing", "Shanghai", "Guangzhou", "Chengdu"]

39. cities[0] = _____
40. cities[3] = _____
41. cities[-1] = _____
42. cities[2] = _____

43. What happens if you write cities[5]? _____

Section D: Coding Tasks (编程任务)

Write and test in Thonny.

15. CLASS REGISTER PROGRAM:

Create a list with 4 student names from our class.

Print the list.

Add one new student using append().

Print the updated list and the total number of students.

[Write your code here]

16. SHOPPING LIST:

Create an empty shopping list.

Use a WHILE loop to ask the user to enter items.

Stop when they type "done".

Print the complete shopping list.

[Write your code here]

17. EXTENSION:

Start with: scores = [78, 65, 92, 88, 70]

Remove the lowest score.

Add a new score of 95.

Print the updated list.

[Write your code here]

WORKSHEET 8B — Traversing Lists, Statistics & Linear Search

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Multiple Choice (选择题)

44. Which loop is best for processing every element in a list?

a) WHILE loop b) FOR loop c) IF statement d) Both a and b

2. To find the largest value in a list, you should:

a) Set max = 0 at the start

b) Set max = first element, then compare with each item

c) Use the largest() function

d) Sort the list and take the last item only

3. The "accumulator pattern" means:

a) Adding money to a bank account

b) Building up a total in a variable

c) Storing data in a list

d) Counting how many items are in a list

4. Linear search is called "linear" because:

a) It only works with numbers in a line

b) It checks each item one by one in order

c) It draws a line on the screen

d) It is faster than binary search

Section B: Trace Table (跟踪表)

Trace this program. Complete the table.

```
marks = [72, 85, 60, 91, 78]
highest = marks[0]
for mark in marks:
    if mark > highest:
        highest = mark
    print(f"mark={mark}, highest={highest}")
```

Iteration	mark	highest (after)
Start	—	
1	72	
2	85	
3	60	
4	91	
5	78	

Section C: Write the Algorithm (写算法)

Write pseudocode OR Python code for each task.

45. Calculate the average of a list of numbers called scores.
46. Count how many items in a list are equal to the target value.

7. Linear search: Check if a name is in the class register.

Section D: Coding Challenge (编程挑战)

8. STUDENT MARKS PROGRAM:

Write a complete Python program that:

a) Creates a list of 5 student marks (use input() or set values)

b) Calculates and prints the total marks

c) Calculates and prints the average mark

d) Finds and prints the highest mark

e) Finds and prints the lowest mark

f) Counts and prints how many students passed (mark >= 50)

[Write your code here or in Thonny]

9. EXTENSION — DISTINCTION COUNTER:

Add to your program: count how many students got Distinction (>= 85)

and how many got Merit (65-84). Print both counts.

[Write your code here or in Thonny]

ANSWER KEY — WEEK 8 WORKSHEETS

WORKSHEET 8A ANSWERS:

Section A:

47. list / 列表
48. 0 (zero)
49. append()
50. len()
51. remove()

Section B:

52. banana
53. [10, 20, 99, 40]
54. ["red", "yellow", "green", "blue"]

Explanation: append("blue") adds to end → ["red","green","blue"]

insert(1, "yellow") puts "yellow" at index 1 → ["red","yellow","green","blue"]

55. ["pen", "book"]

2

Section C:

56. "Shenzhen"
57. "Guangzhou"
58. "Chengdu"
59. "Shanghai"

14. IndexError: list index out of range

(Python will crash with an error message)

Section D:

15. Sample answer:

```
register = ["Li Wei", "Wang Fang", "Zhang Ming", "Liu Hua"]
print(register)
register.append("Chen Jie")
print(register)
print(f"Total students: {len(register)}")
```

16. Sample answer:

```
shopping = []
item = input("Enter item (or 'done'): ")
while item != "done":
    shopping.append(item)
    item = input("Enter item (or 'done'): ")
print(f"Shopping list: {shopping}")
```

17. Sample answer:

```
scores = [78, 65, 92, 88, 70]
scores.remove(65)  # or scores.pop(1)
scores.append(95)
print(scores)
Output: [78, 92, 88, 70, 95]
```

WORKSHEET 8B ANSWERS:

Section A:

60. b) FOR loop
61. b) Set max = first element, then compare with each item
62. b) Building up a total in a variable
63. b) It checks each item one by one in order

Section B:

Iteration	mark	highest (after)
Start	—	72
1	72	72

2	85	85
3	60	85
4	91	91
5	78	91

Section C:

5. Pseudocode:

```
total ← 0
FOR EACH score IN scores DO
    total ← total + score
ENDFOR
average ← total / LENGTH(scores)
OUTPUT average
```

Python:

```
total = 0
for score in scores:
    total = total + score
average = total / len(scores)
print(average)
```

6. Pseudocode:

```
count ← 0
FOR EACH item IN list DO
    IF item = target THEN
        count ← count + 1
    ENDIF
ENDFOR
OUTPUT count
```

Python:

```
count = 0
for item in list:
    if item == target:
        count = count + 1
print(count)
```

7. Pseudocode:

```
found ← FALSE
FOR i ← 0 TO LENGTH(register) - 1 DO
    IF register[i] = targetName THEN
        OUTPUT "Found at position", i
        found ← TRUE
    ENDIF
ENDFOR
IF found = FALSE THEN
    OUTPUT "Not found"
ENDIF
Python:
found = False
for i in range(len(register)):
    if register[i] == targetName:
        print(f"Found at position {i}")
        found = True
if not found:
    print("Not found")
```

Section D:

8. Sample answer:

```
marks = []
for i in range(5):
    mark = int(input(f"Enter mark for student {i+1}: "))
    marks.append(mark)
total = 0
highest = marks[0]
lowest = marks[0]
pass_count = 0
for mark in marks:
    total = total + mark
    if mark > highest:
        highest = mark
    if mark < lowest:
```

```
            lowest = mark
        if mark >= 50:
            pass_count = pass_count + 1
    average = total / len(marks)
    print(f"Total: {total}")
    print(f"Average: {average}")
    print(f"Highest: {highest}")
    print(f"Lowest: {lowest}")
    print(f"Passed: {pass_count}")
```

9. Extension:

```
    distinction = 0
    merit = 0
    for mark in marks:
        if mark >= 85:
            distinction = distinction + 1
        elif mark >= 65:
            merit = merit + 1
    print(f"Distinctions: {distinction}")
    print(f"Merits: {merit}")
```

CIE-Style Exam Question (Bonus, 5 marks):

"A school stores exam marks for 200 students in a 1D array called Marks.

The program needs to find how many students achieved a mark of 90 or above.

Write pseudocode for this algorithm."

Mark Scheme:

- Initialise counter to 0 [1]
- Loop through all elements of Marks [1]
- Check if current mark >= 90 [1]
- If true, increment counter [1]
- Output counter after loop [1]

Sample Answer:

```
 count ← 0
 FOR i ← 0 TO 199 DO
    IF Marks[i] >= 90 THEN
       count ← count + 1
    ENDIF
```

ENDFOR

OUTPUT count

WEEK 9: 2D ARRAYS / NESTED LISTS

LESSON 9.1 — 2D Arrays: Rows and Columns

Topic: 2D arrays — rows and columns, creating nested lists, accessing elements with [row][col]

Duration: 40 minutes

Lesson Type: Programming / Conceptual

Syllabus Ref: 2.2.3 (2D Arrays)

Learning Objective:

By the end of this lesson, students will be able to create 2D arrays (nested lists) in Python, understand row/column indexing, and access individual elements using double indexing [row][col].

Key Vocabulary:

- 2D array / 二维数组 (èr wéi shù zǔ) — array with rows and columns
- nested list / 嵌套列表 (qiàn tào liè biǎo) — list inside another list
- row / 行 (háng) — horizontal line of data
- column / 列 (liè) — vertical line of data
- element / 元素 (yuán sù) — single data item
- matrix / 矩阵 (jǔ zhèn) — mathematical name for 2D array
- grid / 网格 (wǎng gé) — visual representation of 2D data

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| Display a cinema seating plan on board:
| Col 0 Col 1 Col 2 Col 3
| Row 0 [A] [B] [C] [D]
| Row 1 [E] [F] [G] [H]
| Row 2 [I] [J] [K] [L]
| "If seat F is Row 1, Col 1, what is seat K?"
| Answer: Row 2, Col 2. Introduce: "This is a 2D array!"

3-12 min | DIRECT INSTRUCTION

| Write title: "2D Arrays — Tables of Data"
| CONCEPT 1: Why 2D arrays?
| "A 1D list is like a single line. A 2D array is like a TABLE
| with rows AND columns."

| 1D list: [10, 20, 30] — one row

| 2D array: Row 0 → [10, 20, 30]

| Row 1 → [40, 50, 60]

| Row 2 → [70, 80, 90]

| CONCEPT 2: Creating nested lists in Python

| timetable = [

| ["Maths", "English", "Science"],

| ["History", "CS", "Chinese"],

| ["PE", "Maths", "Art"]

| Visual on board: draw boxes with row and column labels.

| CONCEPT 3: Accessing elements — DOUBLE INDEX

| timetable[row][column]

| timetable[0][0] → "Maths" (Row 0, Col 0)

| timetable[0][1] → "English" (Row 0, Col 1)

| timetable[1][2] → "Chinese" (Row 1, Col 2)

| timetable[2][0] → "PE" (Row 2, Col 0)

| MEMORY TRICK: "Row first, Column second — like reading

| a book: which PAGE (row), which WORD on the line (col)."

| CONCEPT 4: Common ERRORS

| - timetable[3][0] → IndexError (no row 3!)

| - timetable[0][3] → IndexError (no col 3 in row 0!)

| - Forgetting double brackets: timetable[0,1] → WRONG!

| CONCEPT 5: CINEMA SEATING EXAMPLE

| cinema = [

| ["A1", "A2", "A3", "A4"],

| ["B1", "B2", "B3", "B4"],

| ["C1", "C2", "C3", "C4"]

| print(cinema[1][2]) # "B3"

| cinema[1][2] = "XX" # Mark seat as booked

| print(cinema[1][2]) # "XX"

12-25 min | **GUIDED PRACTICE**

| Task 1 (whole class):

| Given the timetable above, what do these print?

| timetable[2][1] → "Maths"

| timetable[1][0] → "History"

| timetable[0][2] → "Science"

| Students use mini-whiteboards.

| Task 2 (teacher codes live):

| Create a 3x3 grid of numbers and print the diagonal.

```
|  grid = [
|      [1, 2, 3],
|      [4, 5, 6],
|      [7, 8, 9]
|  print(grid[0][0])  # 1
|  print(grid[1][1])  # 5
|  print(grid[2][2])  # 9
```

| Task 3 (pairs):

| "Create a 2D array for student marks in 3 subjects:

| Row 0: [78, 85, 92] (Student 1)

| Row 1: [88, 76, 90] (Student 2)

| Row 2: [65, 92, 80] (Student 3)

| Print student 2's mark in subject 1 (index [1][1])."

| Answer: 76

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 9A.

| Students work on computers:

| - Create 2D arrays for timetable, cinema, or marks

| - Access specific elements

| - Modify elements (book seats, update marks)

| - Draw the grid on paper first, then code

35-40 min | PLENARY + HOMEWORK

| Plenary: Quick-fire questions with cinema grid

| "What is at cinema[0][3]? What about cinema[2][0]?"

| "How would you mark seat C2 as booked?"

| Key question: "Why do we use [row][col] and not [col][row]?"

| Answer: "It's the convention. Think: first pick the row

| (which student), then pick the column (which subject)."

| Homework: Complete WORKSHEET 9A Section D.

| "Draw a 4x4 multiplication table as a 2D array on paper.

| Label all rows and columns with indices."

EAL Support:

- Draw grid on board with colour-coded rows and columns

- Physical: use students standing in rows — "Which row? Which position?"

- Cinema seating is culturally familiar (movie theatres in Shenzhen)

- Bilingual: 行 (háng = row, sounds like "hang" horizontal)

- Hand gesture: horizontal sweep = row, vertical sweep = column

Differentiation:

Emerging:

- Work with provided 2D arrays (don't need to create from scratch)
- Focus on reading/accessing only (not modifying)
- Use visual worksheet with grid drawn and labelled
- Trace specific [row][col] accesses on paper grid

Developing:

- Standard worksheet — create and access 2D arrays
- Modify elements (book cinema seats)
- Create own 3x3 grid
- Combine with FOR loops from Week 7

Secure:

- Extension: Create a tic-tac-toe board using 2D array
- Build a seating plan with user input
- Calculate row sums and column sums manually
- Peer tutor Emerging students
- Challenge: "Create a 5x5 multiplication table using nested loops"

Resources Needed:

- Thonny IDE on all computers

- Projector with grid visuals

- Mini-whiteboards and markers

- WORKSHEET 9A + 9B

- Blank grid paper for drawing 2D arrays

- Colour-coded row/column reference card

LESSON 9.2 — Traversing 2D Arrays: Nested FOR Loops

Topic: Traversing 2D arrays with nested FOR loops, row sums, column sums, finding values in 2D arrays

Duration: 40 minutes

Lesson Type: Programming / Practical

Syllabus Ref: 2.2.3 (2D Arrays)

Learning Objective:

By the end of this lesson, students will be able to traverse all elements of a 2D array using nested FOR loops, calculate row sums and column sums, and search for values in a 2D array.

Key Vocabulary:

- nested loop / 嵌套循环 (qiàn tào xún huán) — loop inside another loop
- outer loop / 外层循环 (wài céng xún huán) — the main loop (usually rows)
- inner loop / 内层循环 (nèi céng xún huán) — loop inside (usually columns)
- row sum / 行和 (háng hé) — total of one row
- column sum / 列和 (liè hé) — total of one column
- transpose / 转置 (zhuǎn zhì) — swap rows and columns

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| "This code uses a loop inside a loop. Predict the output:
| for i in range(3):
| for j in range(2):
| print(i, j)
| How many lines will it print?"
| Answer: 6 lines (3 x 2). Introduce: "Nested loops!"

3-12 min | DIRECT INSTRUCTION

| Write title: "Nested Loops — Processing Tables"
| CONCEPT 1: How nested loops work
| for i in range(3): ← Outer loop: runs 3 times
| for j in range(4): ← Inner loop: runs 4 times EACH
| print("*") ← This line runs 3 x 4 = 12 times!
| Trace on board:
| i=0: j=0, j=1, j=2, j=3 (inner runs 4 times)
| i=1: j=0, j=1, j=2, j=3 (inner runs 4 times)
| i=2: j=0, j=1, j=2, j=3 (inner runs 4 times)
| Total iterations: 3 x 4 = 12
| CONCEPT 2: Traversing a 2D array
| marks = [
| [78, 85, 92],
| [88, 76, 90],
| [65, 92, 80]
| for row in range(len(marks)): # Outer: each row
| for col in range(len(marks[0])): # Inner: each column
| print(marks[row][col])
| CONCEPT 3: Row sums
| for row in range(len(marks)):

```
|       row_total = 0
|       for col in range(len(marks[0])):
|           row_total = row_total + marks[row][col]
|       print(f"Student {row} total: {row_total}")
| CONCEPT 4: Average per student AND per subject
| - Row average = student average (across subjects)
| - Column average = subject average (across students)
| CONCEPT 5: Finding a value in 2D array
|   target = 92
|   found = False
|   for row in range(len(marks)):
|       for col in range(len(marks[0])):
|           if marks[row][col] == target:
|               print(f"Found at row {row}, col {col}")
|               found = True
```

12-25 min | **GUIDED PRACTICE**

```
| Task 1 (teacher codes live):
| Print all elements of the marks 2D array with formatting.
|   for row in range(len(marks)):
|       for col in range(len(marks[0])):
|           print(f"{marks[row][col]:4}", end="")
|       print()  # New line after each row
| Output:
|    78  85  92
|    88  76  90
|    65  92  80
| Task 2 (pairs):
| "Calculate the average mark for Subject 1 (column 1)."
| Answer:
|   total = 0
|   for row in range(len(marks)):
|       total = total + marks[row][1]
|   average = total / len(marks)
| Task 3 (teacher codes):
| Complete program: input marks for 3 students, 3 subjects,
| then calculate and display student averages and subject averages.
```

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 9B.

| Students build the MULTI-SUBJECT MARKS PROGRAM:

| - Create a 2D array for 3 students x 3 subjects

| - Calculate each student's average

| - Calculate each subject's average

| - Find highest mark and its position

| Extension: Find which student has the highest average

35-40 min | PLENARY + HOMEWORK

| Plenary: "3-2-1"

| - 3 things you learned about 2D arrays

| - 2 things you found challenging

| - 1 question you still have

| Key question: "If a 2D array has 5 rows and 4 columns, and

| the inner loop takes 2 lines of code, how many times do those

| 2 lines run?"

| Answer: 5 x 4 = 20 times.

| Homework: Complete WORKSHEET 9B.

| "Create a 4x4 times table using nested loops. Save as

| timetable.py and email screenshot."

EAL Support:

- "Loop inside a loop" = 循环里面还有循环 — visual: draw concentric circles

- Outer loop = the "big" loop, inner loop = the "small" loop inside

- Use nested boxes diagram: draw boxes inside boxes

- Number each iteration on the board when tracing

- Connect to maths: rows and columns in a matrix

Differentiation:

Emerging:

- Pre-written 2D array; student only writes the nested loop to print it
- Focus on understanding the pattern (outer=rows, inner=columns)
- Use trace table to follow execution
- Row sums only (not column sums)

Developing:

- Standard worksheet — row sums, column sums
- Calculate averages per student
- Format output nicely
- Find highest mark in the 2D array

Secure:

- Extension: Calculate column averages (harder — need to fix column, vary row)
- Find the position of highest and lowest marks
- Create a transposed version of the array
- Peer tutor Emerging students
- Challenge: "Write a function that checks if a 2D array is a magic square (all rows, columns, diagonals sum to same value)"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- WORKSHEET 9B

- 2D array trace table templates

- Nested loop structure poster

WORKSHEET 9A — 2D Arrays: Creating and Accessing

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Vocabulary (词汇)

1. A _____ array has both rows and columns. (二维)

64. In Python, a 2D array is implemented as a _____ list. (嵌套)
65. To access an element, use two indices: first the _____, then the _____. (行, 列)

4. A _____ is a loop inside another loop. (嵌套循环)

Section B: Refer to this 2D array (参考以下二维数组):

```
timetable = [
    ["Maths",  "English",  "Science"],
    ["History", "CS",      "Chinese"],
    ["PE",     "Maths",    "Art"]
]
```

Write the output of each statement (写出每个语句的输出):

66. print(timetable[0][0]) → _____
67. print(timetable[1][2]) → _____
68. print(timetable[2][1]) → _____

69. print(timetable[0][2]) → _____
70. print(timetable[2][0]) → _____

10. What will happen if you write timetable[3][0]?

11. Write ONE line of code to change "Art" to "Music".

Section C: Cinema Seating (电影院座位)

The cinema array below shows available (O) and booked (X) seats:

```
cinema = [
   ["O", "O", "X", "O"],
   ["O", "X", "X", "O"],
   ["O", "O", "O", "X"]
]
```

12. How many rows are there? _____

13. How many seats per row? _____

14. What is cinema[1][1]? What does it mean?

71. Write code to mark seat Row 2, Col 3 as booked (change to "X").

16. Write a FOR loop to print all seats in Row 0.

Section D: Coding (编程)

72. Create your own 2D array for a class with 3 students and 4 subjects.
    Each element is a mark (0-100).
    Print the mark for Student 2 (index 1), Subject 3 (index 2).
    Print all marks for Student 0.
    [Write your code here or in Thonny]

18. EXTENSION: A TIC-TAC-TOE board

Create a 3x3 board filled with empty spaces " ".

Allow the user to place "X" at a chosen row and column.

Print the updated board.

[Write your code here or in Thonny]

WORKSHEET 9B — Nested Loops: Traversing 2D Arrays

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Predict the Output (预测输出)

For each nested loop, write ALL lines that will be printed.

73.
```
for i in range(2):
    for j in range(3):
        print(i, j)
```
Output:

74.
```
for row in range(3):
    for col in range(2):
        print("*", end="")
    print()
```
Output:

75.
```
total = 0
for i in range(2):
    for j in range(2):
        total = total + 1
print(total)
```
Output: _____

Section B: Trace Table (跟踪表)

Trace this nested loop. Complete the table.

```
grid = [[3, 5], [2, 8]]
total = 0
for row in range(2):
    for col in range(2):
        total = total + grid[row][col]
        print(f"row={row}, col={col}, value={grid[row][col]}, total={total}")
```

Step	row	col	grid[row][col]	total
Init	—	—	—	0

Section C: Write the Code (写代码)

76. Write a nested loop to print all elements of this 2D array:

```
data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

(Print one number per line)

5. Write code to calculate the sum of Row 1 only.

6. Write code to calculate the average of Column 0.

Section D: Coding Challenge (编程挑战)

7. STUDENT MARKS PROGRAM:

Create a 2D array for 3 students and 3 subjects:

Student 0: [78, 85, 92]

Student 1: [88, 76, 90]

Student 2: [65, 92, 80]

a) Use nested loops to print the array in a grid format.

b) Calculate and print each student's total and average.

c) Calculate and print each subject's average.

d) Find and print the highest mark in the entire array,
with its row and column position.

[Write your code in Thonny. Save as marks2d.py]

8. EXTENSION:

Add input() so the teacher can enter marks for any number of students.

Ask: "How many students?" then "How many subjects?"

Then use nested loops to input each mark.

Calculate and display all averages.

[Write your code in Thonny]

ANSWER KEY — WEEK 9 WORKSHEETS

WORKSHEET 9A ANSWERS:

Section A:

77. 2D / 二维
78. nested / 嵌套
79. row / 行, column / 列
80. nested loop / 嵌套循环

Section B:

81. "Maths"
82. "Chinese"
83. "Maths"
84. "Science"
85. "PE"

10. IndexError: list index out of range (there is no row 3)

86. timetable[2][2] = "Music"

Section C:

87. 3
88. 4
89. "X" — it means the seat is booked
90. cinema[2][3] = "X"
91. for i in range(4):
    print(cinema[0][i])

Output: O O X O

Section D:

17. Sample:

```
marks = [
    [85, 78, 92, 88],
    [76, 90, 85, 79],
    [92, 88, 76, 95]
]
print(marks[1][2])   # 85
for i in range(4):
    print(marks[0][i])
```

18. Sample:

```
board = [[" ", " ", " "], [" ", " ", " "], [" ", " ", " "]]
row = int(input("Enter row (0-2): "))
col = int(input("Enter col (0-2): "))
board[row][col] = "X"
for r in range(3):
    for c in range(3):
        print(f"[{board[r][c]}]", end="")
    print()
```

WORKSHEET 9B ANSWERS:

Section A:

92. 0 0
    0 1

0 2

1 0

1 1

1 2

(6 lines total)

93.

* *

* *

(3 rows of 2 stars each)

94. 4

Explanation: 2 x 2 = 4 iterations, total increments 4 times

Section B:

| Step | row | col | grid[row][col] | total |

| Init | — | — | — | 0 |

| 1 | 0 | 0 | 3 | 3 |

| 2 | 0 | 1 | 5 | 8 |

| 3 | 1 | 0 | 2 | 10 |

| 4 | 1 | 1 | 8 | 18 |

Section C:

95.
```
for row in range(3):
    for col in range(3):
        print(data[row][col])
```
Or using len():
```
for row in range(len(data)):
    for col in range(len(data[0])):
        print(data[row][col])
```
96.
```
total = 0
for col in range(3):
    total = total + data[1][col]
print(total)
# data[1] = [4, 5, 6], sum = 15
```
97.
```
total = 0
for row in range(3):
    total = total + data[row][0]
average = total / 3
print(average)
```

```
  # Column 0: 1 + 4 + 7 = 12, average = 4.0
```

Section D:

7. Complete sample solution:

```
marks = [
    [78, 85, 92],
    [88, 76, 90],
    [65, 92, 80]
]
# a) Print grid
print("Mark Grid:")
for row in range(len(marks)):
    for col in range(len(marks[0])):
        print(f"{marks[row][col]:4}", end="")
    print()
# b) Student totals and averages
print("\nStudent Averages:")
for row in range(len(marks)):
    total = 0
    for col in range(len(marks[0])):
        total = total + marks[row][col]
    average = total / len(marks[0])
    print(f"Student {row}: Total={total}, Average={average:.1f}")
# c) Subject averages
print("\nSubject Averages:")
for col in range(len(marks[0])):
    total = 0
    for row in range(len(marks)):
        total = total + marks[row][col]
    average = total / len(marks)
    print(f"Subject {col}: Average={average:.1f}")
# d) Find highest
highest = marks[0][0]
high_row = 0
high_col = 0
for row in range(len(marks)):
```

```
        for col in range(len(marks[0])):
            if marks[row][col] > highest:
                highest = marks[row][col]
                high_row = row
                high_col = col
    print(f"\nHighest mark: {highest} at row {high_row}, col {high_col}")
```

8. Extension sample:

```
    students = int(input("How many students? "))
    subjects = int(input("How many subjects? "))
    marks = []
    for s in range(students):
        row = []
        for sub in range(subjects):
            m = int(input(f"Mark for student {s}, subject {sub}: "))
            row.append(m)
        marks.append(row)
    # Then same calculation code as above
```

CIE-Style Exam Question (Bonus, 6 marks):

"A school stores marks for 50 students in 5 subjects using a 2D array called Marks[50][5]. Write pseudocode to:

a) Calculate each student's average mark [3 marks]

b) Find which student has the highest average [3 marks]"

Mark Scheme (a):

- Outer loop for each student [1]
- Inner loop to sum marks for that student [1]
- Calculate and store average [1]

Mark Scheme (b):

- Initialise highest average variable [1]
- Compare each student's average [1]
- Track and output student with highest average [1]

Sample Answer:

```
highestAvg ← -1
bestStudent ← 0
FOR s ← 0 TO 49 DO
    total ← 0
```

```
    FOR sub ← 0 TO 4 DO
      total ← total + Marks[s][sub]
    ENDFOR
    avg ← total / 5
    OUTPUT "Student", s, "average:", avg
    IF avg > highestAvg THEN
      highestAvg ← avg
      bestStudent ← s
    ENDIF
  ENDFOR
  OUTPUT "Best student:", bestStudent, "with average", highestAvg
```

WEEK 10: STRING HANDLING

LESSON 10.1 — String Operations: Slicing, Methods & ASCII

Topic: String operations — concatenation, slicing [start:end], length, upper/lower, find(), replace(), split(), ASCII with ord() and chr()

Duration: 40 minutes

Lesson Type: Programming / Conceptual

Syllabus Ref: 2.2.4 (String Handling)

Learning Objective:

By the end of this lesson, students will be able to manipulate strings using slicing, common string methods, and understand ASCII character encoding using ord() and chr().

Key Vocabulary:

- string / 字符串 (zì fú chuàn) — sequence of characters
- concatenation / 连接 (lián jiē) — joining strings together
- slicing / 切片 (qiē piàn) — extracting part of a string
- length / 长度 (cháng dù) — number of characters (len())
- ASCII / ASCII 码 — character encoding standard
- ord() / 字符转数字 — convert character to ASCII number
- chr() / 数字转字符 — convert ASCII number to character
- method / 方法 (fāng fǎ) — function that belongs to an object

Lesson Structure (40 Minutes):

0-3 min | **DO NOW**

| "Given: name = 'Shenzhen'

| Without running code, predict:

| a) len(name) b) name[0] c) name[-1]

| d) name[0:4] e) name.upper()

| Write predictions in notebook."

| Check answers after 2 minutes. Reward correct predictions.

3-12 min | DIRECT INSTRUCTION

| Write title: "String Handling — Text Processing in Python"

| CONCEPT 1: Strings are like lists of characters

| name = "Python"

| Index: P y t h o n

| 0 1 2 3 4 5

| print(name[0]) → "P"

| print(name[3]) → "h"

| print(len(name)) → 6

| CONCEPT 2: String slicing [start:end]

| text = "Shenzhen"

| text[0:4] → "Shen" (start at 0, stop BEFORE 4)

| text[4:8] → "zhen" (start at 4, stop BEFORE 8)

| text[:4] → "Shen" (start from beginning)

| text[4:] → "zhen" (go to end)

| text[:] → "Shenzhen" (whole string)

| MEMORY RULE: "Start is INclusive, End is EXclusive"

| 开始包含，结束不包含

| CONCEPT 3: String methods

| text.upper() → "SHENZHEN" (all capitals)

| text.lower() → "shenzhen" (all lowercase)

| text.find("e") → 2 (first position of "e")

| text.find("z") → 4

| text.find("x") → -1 (not found!)

| text.replace("e", "3") → "Sh3nzh3n"

| text.split("e") → ["Sh", "nzh", "n"]

| CONCEPT 4: Concatenation (+) and repetition (*)

| first = "Hello"

| second = "World"

| print(first + " " + second) → "Hello World"

| print("Hi" * 3) → "HiHiHi"

| CONCEPT 5: ASCII — How computers store characters

| Each character has a number code.

| 'A' = 65, 'B' = 66, ... 'Z' = 90

| 'a' = 97, 'b' = 98, ... 'z' = 122

| '0' = 48, '1' = 49, ... '9' = 57

| In Python:

| ord('A') → 65

| ord('a') → 97

| chr(65) → 'A'

| chr(97) → 'a'

| DEMO: Show that ord('A') + 1 gives 66, chr(66) is 'B'

| "This is how Caesar cipher works!"

12-25 min | GUIDED PRACTICE

| Task 1 (pairs, mini-whiteboards):

| word = "Internationalization"

| a) What is word[0:5]? → "Inter"

| b) What is word[5:10]? → "natio"

| c) What is len(word)? → 20

| d) word.find("z")? → -1

| Task 2 (teacher codes live):

| Build a PASSWORD STRENGTH CHECKER.

```
| password = input("Enter password: ")
| strength = 0
| if len(password) >= 8:
|     strength = strength + 1
| has_upper = False
| has_lower = False
| has_digit = False
| for char in password:
|     if char.isupper(): has_upper = True
|     if char.islower(): has_lower = True
|     if char.isdigit(): has_digit = True
| if has_upper: strength = strength + 1
| if has_lower: strength = strength + 1
| if has_digit: strength = strength + 1
| print(f"Password strength: {strength}/4")
```

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 10A.

| Students complete:

| - String slicing exercises

| - Method practice (find, replace, split)

| - ASCII conversion exercises

| - Password strength checker task

| Teacher circulates, helps with slicing boundaries.

35-40 min | PLENARY + HOMEWORK

| Plenary: "True or False"

| - "text[0:3] includes the character at index 3" → FALSE

| - "find() returns -1 when the substring is not found" → TRUE

| - "'ABC'.lower() changes the original string" → FALSE

| - "ord('0') gives the value 0" → FALSE (it's 48)

| Key question: "Why does find() return -1 for 'not found'

| instead of 0?"

| Answer: "Because 0 is a valid position (the first character)!

| -1 can never be a real position in a string."

| Homework: Complete WORKSHEET 10A.

| "Write a program that takes a name and formats it properly:

| - Remove leading/trailing spaces (strip())

| - Capitalize first letter of each word (title())

| - Print 'Welcome, [Name]!'"

EAL Support:

- Visual: draw string with numbered boxes for each character

- Colour-code: slice start = green, slice end = red (stop before red)

- ASCII table printed on desk card (A-Z, a-z, 0-9)

- "String methods don't change the original" — explain with analogy:

"Calling .upper() is like taking a photo — the original doesn't change"

- Sentence frame: "The slice text[__:__] gives '___'"

Differentiation:

Emerging:

- Focus on basic indexing and len() only
- Pre-filled code for slicing exercises
- ASCII table always visible
- Simple concatenation tasks
- Modified password checker (just check length)

Developing:

- Standard worksheet — all string methods
- Slicing with different start/end values
- Complete password strength checker
- ASCII conversion exercises

Secure:

- Extension: Write a Caesar cipher encoder/decoder
- Implement string reversal using slicing [::-1]
- Check password against common passwords list
- Peer tutor Emerging students
- Challenge: "Write a program that checks if two strings are anagrams"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- WORKSHEET 10A + 10B

- ASCII reference card (printed, bilingual)

- String methods quick reference card

LESSON 10.2 — String Processing: Counting, Reversing & Palindromes

Topic: Counting characters, reversing strings, palindrome checking, converting between strings and lists

Duration: 40 minutes

Lesson Type: Programming / Practical

Syllabus Ref: 2.2.4 (String Handling)

Learning Objective:

By the end of this lesson, students will be able to write programs that count characters, reverse strings, check for palindromes, and convert between strings and lists for text analysis.

Key Vocabulary:

- reverse / 反转 (fǎn zhuǎn) — turn backwards

- palindrome / 回文 (huí wén) — reads same forwards and backwards

- character count / 字符计数 (zì fú jì shù) — how many of each letter

- word count / 字数统计 (zì shù tǒng jì) — number of words

- split / 分割 (fēn gē) — break string into parts

- join / 连接 (lián jiē) — combine list into string

- text analysis / 文本分析 (wén běn fēn xī) — process text to find info

Lesson Structure (40 Minutes):

0-3 min | **DO NOW**

| "What is a palindrome? Can you name one in English?
| Can you name one in Chinese?"
| Examples: "radar", "level", "madam", "racecar"
| Chinese: 上海自来水来自海上, 人人为我我为人人
| "Today we write a program to check!"

3-12 min | **DIRECT INSTRUCTION**

| Write title: "Text Processing — Advanced String Operations"
| CONCEPT 1: Counting characters in a string
| text = "Shenzhen is a beautiful city"
| count = 0
| for char in text:
| if char == "e":
| count = count + 1
| print(f"'e' appears {count} times")
| CONCEPT 2: Reversing a string (3 ways)
| Method 1: Slicing (easiest!)
| word = "Python"
| reversed = word[::-1] → "nohtyP"
| Explain: [start:end:step] with step=-1 means go backwards
| Method 2: Loop (for understanding)
| reversed = ""
| for char in word:
| reversed = char + reversed
| CONCEPT 3: Palindrome checker
| word = input("Enter a word: ")
| word = word.lower() # Make lowercase
| word = word.replace(" ", "") # Remove spaces
| if word == word[::-1]:
| print("It's a palindrome!")
| else:
| print("Not a palindrome")
| CONCEPT 4: String ↔ List conversion
| # String to list (split)
| sentence = "Hello world from Shenzhen"
| words = sentence.split(" ")

| → ["Hello", "world", "from", "Shenzhen"]

| # List to string (join)

| words = ["Hello", "world"]

| sentence = " ".join(words)

| → "Hello world"

| CONCEPT 5: TEXT ANALYSIS PROGRAM

| Given a paragraph, find:

| - Word count: len(text.split())

| - Sentence count: count periods (.)

| - Character count: len(text)

| - Average word length: total letters / word count

12-25 min | **GUIDED PRACTICE**

| Task 1 (teacher codes live):

| Complete palindrome checker — test with:

| "radar" → palindrome, "hello" → not, "A man a plan a canal

| Panama" → palindrome (after removing spaces!)

| Task 2 (pairs):

| "Write code to count vowels AND consonants in a word."

| word = input("Enter word: ")

| vowels = 0

| consonants = 0

| for char in word.lower():

| if char in "aeiou":

| vowels = vowels + 1

| elif char.isalpha():

| consonants = consonants + 1

| Task 3 (teacher codes):

| Text analysis program skeleton — students complete parts.

| text = input("Enter a paragraph: ")

| words = text.split()

| word_count = len(words)

| sentence_count = text.count(".")

| print(f"Words: {word_count}, Sentences: {sentence_count}")

25-35 min | **INDEPENDENT PRACTICE**

| Distribute WORKSHEET 10B.

| Students choose tasks:

| A) Complete text analysis program

| B) Write a palindrome checker for phrases (ignore spaces/punctuation)

| C) Extension: Pig Latin converter (move first letter to end + "ay")

| Teacher circulates, debugs slicing errors.

35-40 min | PLENARY + HOMEWORK

| Plenary: "One thing you learned, one thing you want to practice"

| Showcase: 2-3 students show their programs.

| Key question: "Why is word[::-1] such a powerful trick?

| What does each : mean?"

| Answer: "[start:end:step]. Empty start/end means whole string.

| Step=-1 means go backwards."

| Homework: Complete WORKSHEET 10B.

| "Write a program that takes a sentence and prints each word

| on a separate line, with its length."

| Example: "Hello world" → "Hello: 5", "world: 5"

EAL Support:

- "Palindrome" = 回文 (hui wen = "returning text") — show visual

- Draw arrow going forward and backward over the same word

- String slicing: physical demonstration with paper strip

- Split/join: use scissors (cut words apart) and glue (join together)

- Bilingual word list for text analysis terms

Differentiation:

Emerging:

- Complete guided code with fill-in-the-blanks
- Focus on counting characters and basic reversal
- Use [::-1] without needing to explain fully
- Simple palindrome checker (single words only)

Developing:

- Standard worksheet — complete text analysis program
- Palindrome checker with spaces removed
- String/list conversion
- Word count and sentence count

Secure:

- Extension: Pig Latin converter
- Advanced text analysis (frequency of each letter)
- String compression (count consecutive characters: "aaabbc" → "a3b2c1")
- Peer tutor Emerging students
- Challenge: "Write a program that suggests spelling corrections

(check if two words differ by one letter)"

Resources Needed:

- Thonny IDE on all computers
- Projector for live coding
- Mini-whiteboards and markers
- WORKSHEET 10B
- Paper strips for physical string demonstration
- Bilingual glossary for text processing terms

WORKSHEET 10A — String Operations: Slicing, Methods & ASCII

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: String Slicing (字符串切片)

Given: city = "Shenzhen"

98. city[0:4] = _____
99. city[4:8] = _____
100. city[:3] = _____
101. city[5:] = _____
102. city[::2] = _____ (every 2nd character)
103. city[::-1] = _____ (reversed)

Given: word = "programming"

104. word[0:4] = _____
105. word[3:7] = _____
106. word[-3:] = _____ (last 3 characters)
107. word[::3] = _____ (every 3rd character)

Section B: String Methods (字符串方法)

Given: text = " Hello Shenzhen "

108. text.strip() = _____
109. text.upper() = _____
110. text.lower() = _____
111. text.find("S") = _____
112. text.replace("Shenzhen", "World") = _____
113. text.split() = _____

Given: email = "student@vpa.edu.cn"

114. email.find("@") = _____
115. email[email.find("@")+1:] = _____

(What does this give you? _____)

Section C: ASCII Values (ASCII 码)

116. ord('A') = _____ 20. ord('Z') = _____
117. ord('a') = _____ 22. ord('0') = _____

118. chr(66) = _____ 24. chr(100) = _____
119. chr(49) = _____

26. What is ord('C') - ord('A')? _____

27. What letter is chr(ord('X') + 2)? _____

Section D: Coding — Password Strength Checker (编程)

28. Complete this password strength checker in Thonny:

```
password = input("Enter password: ")
score = 0
# Check 1: Length at least 8 characters
if _____:
    score = score + 1
# Check 2: Contains uppercase letter
has_upper = False
for char in password:
    if _____:
        has_upper = True
if has_upper:
    score = score + 1
# Check 3: Contains lowercase letter
has_lower = False
for char in password:
    if _____:
        has_lower = True
if has_lower:
    score = score + 1
# Check 4: Contains digit
has_digit = False
for char in password:
    if _____:
        has_digit = True
if has_digit:
    score = score + 1
print(f"Password strength: {score}/4")
```

Test with these passwords:

- "abc" → score: _____
- "Abcdef1" → score: _____
- "Abcdefg1" → score: _____
- "Abcdef1!" → score: _____

WORKSHEET 10B — String Processing: Palindromes & Text Analysis

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Reverse & Palindrome Check (反转与回文)

120. Write ONE line of code to reverse string s: _____

2. Complete the palindrome checker:

```
word = input("Enter a word: ")
word = word.lower()
reversed_word = _____
if _____:
    print("Palindrome!")
else:
    print("Not a palindrome")
```

3. Test your checker with these words:

Word	Expected Result	Your Result
radar	Palindrome	
hello	Not palindrome	
Level	Palindrome	
Aba	Palindrome	
computer	Not palindrome	

Section B: String-List Conversion (字符串与列表转换)

Given: sentence = "The quick brown fox"

121. words = sentence.split() → words = _____

5. Print the second word: _____

6. Join the words back with "-":

result = "-".join(words) → result = _____

Given: data = "87,92,78,95,88"

7. How do you convert this to a list of numbers?

Section C: Text Analysis (文本分析)

8. Complete this text analysis program:

```
text = input("Enter a paragraph: ")
# Word count
words = text._____
word_count = _____
print(f"Words: {word_count}")
# Sentence count (count full stops)
sentence_count = text._____
print(f"Sentences: {sentence_count}")
# Character count (excluding spaces)
char_count = 0
for char in text:
    if _____:
        char_count = char_count + 1
print(f"Characters (no spaces): {char_count}")
# Average word length
total_letters = 0
for word in words:
    total_letters = total_letters + _____
avg_length = total_letters / word_count
print(f"Average word length: {avg_length:.1f}")
```

Section D: Coding Challenge (编程挑战)

9. Write a program that:

a) Asks the user to enter a sentence

b) Prints the sentence reversed (word by word)

c) Prints the longest word

Example: "I love Computer Science"

→ "Science Computer love I"

→ Longest word: "Computer"

[Write your code in Thonny]

10. EXTENSION — CAESAR CIPHER:

Write a program that encodes a message using a Caesar cipher.

Shift each letter by 3 positions.

"ABC" → "DEF", "XYZ" → "ABC" (wrap around!)

Hint: Use ord() and chr().

[Write your code in Thonny]

ANSWER KEY — WEEK 10 WORKSHEETS

WORKSHEET 10A ANSWERS:

Section A:

122. "Shen"
123. "zhen"
124. "She"
125. "en"
126. "Shze" (indices 0,2,4,6 → S,h,z,e)
127. "nehznehS"
128. "prog"
129. "gram"
130. "ing"
131. "pgmi" (word[0]+word[3]+word[6]+word[9] = "p"+"g"+"m"+"i")

Section B:

132. "Hello Shenzhen" (spaces removed from both ends)
133. " HELLO SHENZHEN "
134. " hello shenzhen "
135. 8 (position of 'S' in " Hello Shenzhen ")
136. " Hello World "
137. ["Hello", "Shenzhen"] (split on whitespace by default)
138. 7
139. "vpa.edu.cn" — this extracts the domain part of the email

Section C:

140. 65 20. 90
141. 97 22. 48
142. 'B' 24. 'd'
143. '1'
144. 2 (67 - 65 = 2)
145. 'Z' (ord('X')=88, 88+2=90, chr(90)='Z')

Section D:

28.

if len(password) >= 8:

if char.isupper():

if char.islower():

if char.isdigit():

Test results:

- "abc" → score: 1 (lowercase only)
- "Abcdef1" → score: 3 (upper, lower, digit — but length is 7)
- "Abcdefg1" → score: 4 (length 8, upper, lower, digit)
- "Abcdef1!" → score: 4 (same, the '!' doesn't add to our checks)

WORKSHEET 10B ANSWERS:

Section A:

146. s[::-1]
147. reversed_word = word[::-1]

if word == reversed_word:

3.

Word	Expected Result
radar	Palindrome
hello	Not palindrome
Level	Palindrome
Aba	Palindrome
computer	Not palindrome

Section B:

148. ["The", "quick", "brown", "fox"]
149. print(words[1]) → "quick"
150. "The-quick-brown-fox"
151. parts = data.split(",")

numbers = []

for part in parts:

numbers.append(int(part))

numbers = [87, 92, 78, 95, 88]

Section C:

8.

words = text.split()

word_count = len(words)

sentence_count = text.count(".")

if char != " ": # or: if not char.isspace():

total_letters = total_letters + len(word)

Section D:

9. Sample solution:

```
sentence = input("Enter a sentence: ")
words = sentence.split()
# Reverse word order
reversed_words = words[::-1]
reversed_sentence = " ".join(reversed_words)
print(reversed_sentence)
# Find longest word
longest = words[0]
for word in words:
    if len(word) > len(longest):
        longest = word
print(f"Longest word: {longest}")
```

10. Caesar cipher sample:

```
message = input("Enter message (uppercase only): ")
shift = 3
encoded = ""
for char in message:
    if char.isalpha():
        code = ord(char) + shift
        if code > ord('Z'):
            code = code - 26
        encoded = encoded + chr(code)
    else:
        encoded = encoded + char
print(f"Encoded: {encoded}")
```

CIE-Style Exam Question (Bonus, 4 marks):

"A program needs to check if a password contains at least one digit.

The password is stored in the variable Password.

Write pseudocode for this check."

Mark Scheme:

- Initialise hasDigit to FALSE [1]
- Loop through each character in Password [1]
- Check if character is a digit [1]

- Set hasDigit to TRUE if digit found [1]

Sample Answer:

```
hasDigit ← FALSE
FOR i ← 0 TO LENGTH(Password) - 1 DO
    IF Password[i] >= '0' AND Password[i] <= '9' THEN
        hasDigit ← TRUE
    ENDIF
ENDFOR
```

WEEK 11: FUNCTIONS

LESSON 11.1 — Introduction to Functions: Defining, Calling, Parameters & Return

Topic: What is a function? Why use functions? Defining functions (def). Parameters and arguments. Return values. Built-in vs user-defined.

Duration: 40 minutes

Lesson Type: Programming / Conceptual

Syllabus Ref: 2.2.5 (Functions/Procedures)

Learning Objective:

By the end of this lesson, students will be able to define and call functions with parameters and return values, explain the benefits of using functions, and distinguish between built-in and user-defined functions.

Key Vocabulary:

- function / 函数 (hán shù) — named block of reusable code
- define / 定义 (dìng yì) — create a function
- call / 调用 (diào yòng) — execute a function
- parameter / 参数 (cān shù) — variable in function definition
- argument / 实参 (shí cān) — actual value passed to a function
- return value / 返回值 (fǎn huí zhí) — value sent back from a function
- built-in / 内置 (nèi zhì) — already provided by Python
- user-defined / 用户自定义 (yòng hù zì dìng yì) — created by programmer
- reusable / 可复用 (kě fù yòng) — can be used multiple times

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| "You need to calculate the area of 3 rectangles:
| width=5, height=3
| width=7, height=4
| width=10, height=2

| Write the code WITHOUT using a function."
| Students write 3 separate calculations.
| "What if I asked for 100 rectangles? There is a better way!"

3-12 min | DIRECT INSTRUCTION

| Write title: "Functions — Write Once, Use Many Times"
| CONCEPT 1: What is a function?
| "A function is a NAMED block of code that does ONE job.
| You define it once, then CALL it whenever you need it."
| Real-world analogy: "A blender is a function. You put
| ingredients IN, it does work, and gives you a smoothie OUT.
| You don't rebuild the blender every time you want a drink!"
| CONCEPT 2: Defining a function in Python

```
|   def greet():            ← def = "define"
|       print("Hello!")     ← indented body
|   greet()                 ← CALL the function
|   greet()                 ← Call it again!
```

| CONCEPT 3: Parameters and Arguments

```
|   def greet(name):        ← name is a PARAMETER (placeholder)
|       print(f"Hello, {name}!")
|   greet("Alice")          ← "Alice" is the ARGUMENT
|   greet("Bob")            ← "Bob" is another argument
|   → "Hello, Alice!"
|   → "Hello, Bob!"
```

| CONCEPT 4: Return values

```
|   def add(a, b):          ← Two parameters
|       result = a + b
|       return result       ← Send value back
|   total = add(5, 3)       ← total becomes 8
|   print(total)            → 8
```

| CRITICAL: return ENDS the function immediately!
| Code after return does NOT execute.
| CONCEPT 5: Built-in vs User-defined
| Built-in: print(), len(), input(), range(), int()
| → Python gives you these for free
| User-defined: area(), greet(), add()
| → YOU create these for your program's needs

| BENEFITS OF FUNCTIONS:

| 1. Reusability — write once, use many times

| 2. Organisation — code is easier to read

| 3. Easier testing — test one function at a time

| 4. Less repetition — no copy-paste

12-25 min | GUIDED PRACTICE

| Task 1 (teacher codes live):

| Write a function to calculate area of a rectangle.

```
|   def area_rectangle(width, height):
|       area = width * height
|       return area
|   rect1 = area_rectangle(5, 3)
|   rect2 = area_rectangle(7, 4)
|   print(f"Areas: {rect1}, {rect2}")
```

| Task 2 (pairs, mini-whiteboards):

| "Write a function called area_circle(radius) that returns

| the area of a circle. Use 3.14 for pi."

| Answer:

```
|   def area_circle(radius):
|       return 3.14 * radius * radius
```

| Task 3 (teacher codes):

| Write area_triangle(base, height).

```
|   def area_triangle(base, height):
|       return 0.5 * base * height
```

| Then build a MENU program:

```
|   print("1. Rectangle  2. Circle  3. Triangle")
|   choice = input("Choose: ")
|   if choice == "1":
|       w = float(input("Width: "))
|       h = float(input("Height: "))
|       print(area_rectangle(w, h))
|   elif choice == "2":
|       r = float(input("Radius: "))
|       print(area_circle(r))
|   elif choice == "3":
|       b = float(input("Base: "))
```

| h = float(input("Height: "))

| print(area_triangle(b, h))

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 11A.

| Students write functions for:

| - area_rectangle(w, h)

| - area_circle(r)

| - area_triangle(b, h)

| Test each function with different values.

| Build a menu-driven area calculator.

| Teacher circulates, checks function syntax.

35-40 min | PLENARY + HOMEWORK

| Plenary: "Function or Not?"

| - print("Hello") → Built-in function [YES]

| - len(my_list) → Built-in function [YES]

| - x = 5 + 3 → Not a function [NO]

| - area(4, 5) → User-defined function [YES]

| Key question: "What is the difference between a parameter

| and an argument?"

| Answer: "Parameter is the placeholder in the definition.

| Argument is the actual value you pass when calling."

| Homework: Complete WORKSHEET 11A.

| "Write a function called celsius_to_fahrenheit(c) that

| converts Celsius to Fahrenheit using F = C × 9/5 + 32.

| Test with: 0°C → 32°F, 100°C → 212°F, 37°C → 98.6°F"

EAL Support:

- Function analogy: blender (input → process → output)
- Parameter vs argument: "Parameter = empty box label, Argument = what you put IN the box"
- Visual diagram: function as a machine with input and output
- Colour-code: def keyword (blue), function name (green), parameters (orange), return (purple)
- Bilingual: 函数 (hán shù) = function, 参数 (cān shù) = parameter

Differentiation:

Emerging:

- Fill-in-the-blanks function definitions

- Functions with only one parameter
- Pre-written function calls; student writes function body only
- Use visual function machine diagram
- Focus on functions WITHOUT return (print output instead)

Developing:

- Standard worksheet — define and call functions
- Functions with return values
- Multiple parameters
- Build the menu-driven calculator

Secure:

- Extension: Functions calling other functions
- Add validation (check for negative inputs)
- Calculate volume of shapes (3D)
- Peer tutor Emerging students
- Challenge: "Write a function that returns the nth Fibonacci number"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- WORKSHEET 11A + 11B

- Function machine visual diagram

- Bilingual function terminology card

LESSON 11.2 — Functions with Multiple Parameters, Procedures & Calling Functions

Topic: Functions with multiple parameters, functions without return (procedures), calling functions from other functions, building a calculator

Duration: 40 minutes

Lesson Type: Programming / Practical

Syllabus Ref: 2.2.5 (Functions/Procedures)

Learning Objective:

By the end of this lesson, students will be able to write functions with multiple parameters, understand the difference between functions (with return) and procedures (without return), and call functions from within other functions.

Key Vocabulary:

- procedure / 过程 (guò chéng) — function that does NOT return a value

- void / 无返回值 (wú fǎn huí zhí) — no return value

- nested call / 嵌套调用 (qiàn tào diào yòng) — calling function inside another

- validation / 验证 (yàn zhèng) — checking input is correct

- main program / 主程序 (zhǔ chéng xù) — main part that calls functions

- subroutine / 子程序 (zǐ chéng xù) — another name for function/procedure

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| "Look at these two functions:
| def say_hello(name):
| print(f'Hello, {name}!')
| def double(x):
| return x * 2
| What is the difference? When would you use each?"
| Expected: say_hello prints directly (procedure);
| double returns a value (function).

3-12 min | DIRECT INSTRUCTION

| Write title: "Advanced Functions — Building Bigger Programs"
| CONCEPT 1: Function vs Procedure

| ┌──────────────────────┬──────────────────────┐
	FUNCTION	PROCEDURE
	Has a RETURN value	No return value
	Can be used in an	Stands alone on its own line
	expression	
	Example:	Example:
	total = add(5, 3)	say_hello("Alice")
	print(total + 1)	(cannot use the result)

| In Cambridge pseudocode:
| FUNCTION add(a, b) ← returns value
| RETURN a + b
| END FUNCTION
| PROCEDURE greet(name) ← no return
| OUTPUT "Hello", name
| END PROCEDURE
| CONCEPT 2: Functions calling functions
| def square(x):
| return x * x
| def sum_of_squares(a, b):
| return square(a) + square(b)
| result = sum_of_squares(3, 4) → 9 + 16 = 25
| "sum_of_squares CALLS square. This is how complex programs

| are built — small functions combine into bigger ones."
| CONCEPT 3: Input validation in functions
| def area_rectangle(width, height):
| if width < 0 or height < 0:
| return "Error: dimensions must be positive"
| return width * height
| CONCEPT 4: Default parameters
| def greet(name, greeting="Hello"):
| print(f"{greeting}, {name}!")
| greet("Alice") → "Hello, Alice!"
| greet("Bob", "Hi") → "Hi, Bob!"
| CONCEPT 5: Building a calculator with functions
| Structure:
| def get_number(prompt): ← Input function
| def add(a, b): ← Math functions
| def subtract(a, b):
| def multiply(a, b):
| def divide(a, b):
| def show_menu(): ← Display function
| def main(): ← Main program

12-25 min | GUIDED PRACTICE

| Task 1 (teacher codes live — COMPLETE CALCULATOR):
| def get_number(prompt):
| return float(input(prompt))
| def add(a, b):
| return a + b
| def subtract(a, b):
| return a - b
| def multiply(a, b):
| return a * b
| def divide(a, b):
| if b == 0:
| return "Error: cannot divide by zero"
| return a / b
| def show_menu():
| print("\n1. Add 2. Subtract 3. Multiply 4. Divide 5. Exit")

```
|   def main():
|       while True:
|           show_menu()
|           choice = input("Choose: ")
|           if choice == "5":
|               print("Goodbye!")
|               break
|           num1 = get_number("First number: ")
|           num2 = get_number("Second number: ")
|           if choice == "1": print(add(num1, num2))
|           elif choice == "2": print(subtract(num1, num2))
|           elif choice == "3": print(multiply(num1, num2))
|           elif choice == "4": print(divide(num1, num2))
|   main()
| Task 2 (pairs):
| "Add a power function: power(base, exponent) that uses
|  ** operator. Add it to the menu."
| Answer: def power(a, b): return a ** b
```

25-35 min | INDEPENDENT PRACTICE

```
| Distribute WORKSHEET 11B.
| Students build the CALCULATOR PROGRAM:
| - Write all 4 math functions
| - Add input validation
| - Build the menu system
| - Test with various inputs including divide by zero
| Extension: Add power and modulus operations
```

35-40 min | PLENARY + HOMEWORK

```
| Plenary: Code review
| "Swap with your neighbour. Check their calculator:
|  - Does divide() handle zero?
|  - Is the menu clear?
|  - Are function names descriptive?"
| Key question: "Why is it better to have many small functions
|  than one big block of code?"
| Answer: (1) Easier to test each part
|         (2) Reuse functions in other programs
```

| (3) Easier to find and fix bugs

| (4) Other people can read your code

| Homework: Complete WORKSHEET 11B.

| "Add these functions to your calculator:

| - find_max(a, b, c) — returns the largest of three numbers

| - average(a, b, c) — returns the mean

| Test with your own numbers."

EAL Support:

- Function vs procedure: use colour coding (green=has return, red=no return)

- "Calling a function" = 调用函数 — like calling someone on the phone

- Visual: stack diagram showing function calls

- Use recipe analogy: main() is the main recipe, functions are sub-recipes

- Bilingual code comments for all examples

Differentiation:

Emerging:

- Pre-written calculator skeleton; student fills in function bodies
- Only 2 operations (add, subtract)
- No divide-by-zero handling required
- Focus on understanding function structure

Developing:

- Standard worksheet — complete 4-operation calculator
- Add input validation for divide
- Add power function
- Functions calling functions

Secure:

- Extension: Add memory feature (store result, use in next calculation)
- Scientific calculator mode (sin, cos using math module)
- Add calculation history (list of previous calculations)
- Peer tutor Emerging students
- Challenge: "Write a function that converts between number bases (denary to binary using a function)"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- WORKSHEET 11B

- Calculator program starter code (optional)

- Function call stack diagram

WORKSHEET 11A — Introduction to Functions

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Vocabulary (词汇)

152. A _____ is a named block of reusable code. (函数)
153. The values you pass into a function are called _____. (实参)
154. The variables in a function definition are called _____. (参数)
155. A function sends back a value using the _____ keyword. (返回)
156. _____ functions are provided by Python (like print, len). (内置)

Section B: True or False (判断题)

6. A function must have at least one parameter. T / F

157. A function can have multiple return statements. T / F
158. The return statement ends the function immediately. T / F
159. You must define a function before you can call it. T / F
160. A function can call another function. T / F

Section C: Write the Function (写函数)

161. Write a function called square(n) that returns n squared.
162. Write a function called greet(name) that prints a greeting message.
(This is a PROCEDURE — no return value)
163. Write a function called max_of_two(a, b) that returns the larger number.
164. Write a function called get_perimeter(length, width) that returns the perimeter of a rectangle.

Section D: Function Trace (函数跟踪)

15. Trace this code. What is the final output?

```
def add_ten(x):
    return x + 10
def multiply_by_two(y):
    return y * 2
a = 5
b = add_ten(a)
c = multiply_by_two(b)
print(c)
```

Trace table:

Step	a	b	c
a=5	5		
b=...			

| c=...| | | |

Final output: _____

16. What will this print?

```
def mystery(a, b):
    return a * a + b
print(mystery(3, 4))
print(mystery(2, 5))
```

Output line 1: _____

Output line 2: _____

Section E: Coding Challenge (编程挑战)

17. AREA CALCULATOR with functions:

Write a complete Python program that:

a) Defines: area_rectangle(w, h), area_circle(r), area_triangle(b, h)

b) Asks the user which shape (1, 2, or 3)

c) Asks for the required dimensions

d) Calls the correct function and prints the result

[Write your code in Thonny. Save as area_calculator.py]

18. EXTENSION:

Add a function called volume_cuboid(l, w, h) that returns the volume.

Add it as option 4 in your menu.

[Write your code in Thonny]

WORKSHEET 11B — Functions: Procedures, Nested Calls & Calculator

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Function vs Procedure (函数与过程)

For each function, tick (√) if it is a FUNCTION (has return) or PROCEDURE (no return).

Code Snippet	Function	Procedure
1. def show_title():		
print("My Program")		
2. def get_age():		
return int(input("Age: "))		
3. def calc_average(a, b):		

print((a+b)/2)		
4. def find_max(a, b):		
if a > b:		
return a		
return b		

165. Explain the difference between a function and a procedure:

Section B: Nested Function Calls (嵌套函数调用)

6. Predict the output:

```
def double(x):
    return x * 2
def add_one(x):
    return x + 1
result = add_one(double(3))
print(result)
```

Output: _____

Explanation: _____

7. Predict the output:

```
def square(x):
    return x * x
def sum_squares(a, b):
    return square(a) + square(b)
print(sum_squares(3, 4))
```

Output: _____

Explanation: _____

Section C: Complete the Calculator (完成计算器)

8. Fill in the missing parts:

```
def get_number(prompt):
    return _____
def add(a, b):
    return _____
def subtract(a, b):
    return _____
def multiply(a, b):
```

```
    return _____
def divide(a, b):
    if _____:
        return "Error: cannot divide by zero"
    return _____
def show_menu():
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
    print("4. Divide")
    print("5. Exit")
def main():
    while True:
        choice = input("Choose: ")
        if choice == "5":
            print("Goodbye!")
        num1 = get_number("First number: ")
        num2 = get_number("Second number: ")
        if choice == "1":
            print(_____)
        elif choice == "2":
            print(_____)
        elif choice == "3":
            print(_____)
        elif choice == "4":
            print(_____)
```

Section D: Coding Challenge (编程挑战)

9. ENHANCED CALCULATOR:

Add these functions to your calculator:

a) power(base, exp) — calculates base to the power of exp

b) average_of_three(a, b, c) — returns the average

c) find_max_of_three(a, b, c) — returns the largest

Add them to the menu (options 6, 7, 8).

Test each function with your own values.

[Write your code in Thonny]

10. EXTENSION:

Write a function is_even(n) that returns True if n is even,

False otherwise. Write another function is_prime(n) that returns

True if n is prime. Use these in a number analysis program.

[Write your code in Thonny]

ANSWER KEY — WEEK 11 WORKSHEETS

WORKSHEET 11A ANSWERS:

Section A:

166. function / 函数
167. arguments / 实参
168. parameters / 参数
169. return / 返回

5. Built-in / 内置

Section B:

6. False (functions can have zero parameters)

170. True (but only the first executed return exits the function)

8. True

9. True (Python reads top to bottom)

10. True

Section C:

171. def square(n):
    return n * n
172. def greet(name):
    print(f"Hello, {name}!")
173. def max_of_two(a, b):
    if a > b:
        return a
    else:
        return b

    (Or: return a if a > b else b)
174. def get_perimeter(length, width):
    return 2 * (length + width)

Section D:

15.

| Step | a | b | c |

| a=5 | 5 | | |

| b=... | 5 | 15 | | (b = add_ten(5) = 15)

| c=... | 5 | 15 | 30 | (c = multiply_by_two(15) = 30)

Final output: 30

175. mystery(3, 4) → 3*3 + 4 = 13

mystery(2, 5) → 2*2 + 5 = 9

Output line 1: 13

Output line 2: 9

Section E:

17. Sample solution:

```
def area_rectangle(w, h):
    return w * h
def area_circle(r):
    return 3.14 * r * r
def area_triangle(b, h):
    return 0.5 * b * h
print("1. Rectangle  2. Circle  3. Triangle")
choice = input("Choose: ")
if choice == "1":
    w = float(input("Width: "))
    h = float(input("Height: "))
    print(area_rectangle(w, h))
elif choice == "2":
    r = float(input("Radius: "))
    print(area_circle(r))
elif choice == "3":
    b = float(input("Base: "))
    h = float(input("Height: "))
    print(area_triangle(b, h))
```

176.
```
def volume_cuboid(l, w, h):
    return l * w * h
```

WORKSHEET 11B ANSWERS:

Section A:

Code Snippet	Function	Procedure
1. show_title()		√
2. get_age()	√	
3. calc_average(a, b)		√
4. find_max(a, b)	√	

177. A function returns a value using the return keyword. The calling code can use this value. A procedure does NOT return a value; it performs an action (like printing) and the calling code cannot use a result.

Section B:

6. Output: 7

Explanation: double(3) = 6, then add_one(6) = 7

7. Output: 25

Explanation: square(3) = 9, square(4) = 16, 9 + 16 = 25

Section C:

8. Filled code:

```
def get_number(prompt):
    return float(input(prompt))
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Error: cannot divide by zero"
    return a / b
def show_menu():
    print("1. Add")
    print("2. Subtract")
    print("3. Multiply")
```

```
    print("4. Divide")
    print("5. Exit")
def main():
    while True:
        show_menu()
        choice = input("Choose: ")
        if choice == "5":
            print("Goodbye!")
            break
        num1 = get_number("First number: ")
        num2 = get_number("Second number: ")
        if choice == "1":
            print(add(num1, num2))
        elif choice == "2":
            print(subtract(num1, num2))
        elif choice == "3":
            print(multiply(num1, num2))
        elif choice == "4":
            print(divide(num1, num2))
main()
```

Section D:

9. Sample additions:

```
def power(base, exp):
    return base ** exp
def average_of_three(a, b, c):
    return (a + b + c) / 3
def find_max_of_three(a, b, c):
    if a >= b and a >= c:
        return a
    elif b >= a and b >= c:
        return b
    else:
        return c
```

10. Sample:

```
def is_even(n):
    return n % 2 == 0
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

CIE-Style Exam Question (Bonus, 5 marks):

"Write pseudocode for a function called CalculateAverage that takes three numbers as parameters and returns their average. Then write pseudocode to call this function with the values 10, 20, and 30 and output the result."

Mark Scheme:

- FUNCTION header with correct name and 3 parameters [1]
- Calculate sum of three parameters [1]
- Divide by 3 and RETURN [1]
- Correct function call with arguments 10, 20, 30 [1]
- OUTPUT the returned value [1]

Sample Answer:

```
FUNCTION CalculateAverage(a, b, c)
    sum ← a + b + c
    RETURN sum / 3
END FUNCTION
result ← CalculateAverage(10, 20, 30)
OUTPUT result
```

WEEK 12: SCOPE & MODULAR PROGRAMMING

LESSON 12.1 — Local vs Global Variables & Scope Rules

Topic: Local vs global variables, variable scope rules in Python, why global variables should be avoided, passing data via parameters

Duration: 40 minutes

Lesson Type: Programming / Conceptual

Syllabus Ref: 2.2.5 (Scope of Variables)

Learning Objective:

By the end of this lesson, students will be able to explain the difference between local and global variables, predict variable scope in given code, and write functions that pass data via parameters rather than global variables.

Key Vocabulary:

- scope / 作用域 (zuò yòng yù) — where a variable can be accessed
- local variable / 局部变量 (jú bù biàn liàng) — only inside a function
- global variable / 全局变量 (quán jú biàn liàng) — accessible everywhere
- lifetime / 生命周期 (shēng mìng zhōu qī) — how long a variable exists
- parameter passing / 参数传递 (cān shù chuán dì) — sending data to functions
- side effect / 副作用 (fù zuò yòng) — unexpected change to data
- encapsulation / 封装 (fēng zhuāng) — hiding data inside functions

Lesson Structure (40 Minutes):

0-3 min | DO NOW

```
| Display this code:
|   x = 10
|   def my_func():
|       x = 5
|       print(x)
|   my_func()
|   print(x)
| "What will the two print statements output? Why are they
|  different?"
| Answer: prints 5 then 10. The x inside the function is
| DIFFERENT from the x outside. This is SCOPE.
```

3-12 min | DIRECT INSTRUCTION

```
| Write title: "Variable Scope — Where Does Your Data Live?"
| CONCEPT 1: What is scope?
| "Scope is the RULE about where a variable can be used.
|  Think of it like a house: variables inside a function
|  are like furniture in a bedroom — you can only use them
|  when you're IN that room!"
| CONCEPT 2: Local variables
|   def calculate():
|       result = 10      ← LOCAL: only exists inside calculate()
```

```
|      print(result)   ← OK: we're inside the function
|   calculate()
|   print(result)      ← ERROR! result doesn't exist here!
| Key rule: Variables created INSIDE a function are LOCAL.
| They are created when the function starts and DESTROYED
| when the function ends.
| CONCEPT 3: Global variables
|   score = 100         ← GLOBAL: created outside all functions
|   def show_score():
|      print(score)    ← OK: can READ global variables
|   show_score()       → 100
| CONCEPT 4: THE PROBLEM with global variables
|   total = 0           ← Global variable
|   def add_to_total(x):
|      total = total + x   ← ERROR! Python thinks total is local!
| "This causes an UnboundLocalError! Python sees 'total =' and
|  decides total must be LOCAL. But local total hasn't been
|  assigned yet!"
| THE FIX: Pass data via parameters!
|   def add_to_total(current, x):
|      return current + x
|   total = 0
|   total = add_to_total(total, 5)
|   total = add_to_total(total, 3)
|   print(total)       → 8
| CONCEPT 5: Why avoid global variables?
| 1. ANY function can change them → hard to track bugs
| 2. Functions depend on external data → not reusable
| 3. Hard to test — you need to know ALL global state
| 4. In large programs, name conflicts are common
| GOLDEN RULE: "Pass data IN through parameters.
|               Pass data OUT through return values."
|              数据通过参数传入，通过返回值传出。
| CONCEPT 6: Global keyword (use sparingly!)
|   count = 0
|   def increment():
```

| global count

| count = count + 1

| "The global keyword tells Python: 'I really mean the GLOBAL

| variable, not a local one.' But avoid this pattern!

| Better: def increment(c): return c + 1"

12-25 min | GUIDED PRACTICE

| Task 1 (whole class):

| Predict the output:

| def func_a():

| x = 10

| print(x)

| def func_b():

| x = 20

| print(x)

| func_a() → 10

| func_b() → 20

| func_a() → 10

| "Each function has its OWN x. They don't interfere!"

| Task 2 (pairs, mini-whiteboards):

| "Fix this code using parameters instead of global variables:"

| total = 0 ← DON'T use global

| def add_score(): ← Should take a parameter

| global total ← BAD!

| mark = int(input())

| total = total + mark

| Answer:

| def add_score(current_total, mark):

| return current_total + mark

| total = 0

| total = add_score(total, 85)

| total = add_score(total, 92)

| Task 3 (teacher codes live):

| Refactor a monolithic program into functions with proper scope.

| Start with messy code using global variables, then refactor.

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 12A.

| Students:
| - Predict scope of variables in given code
| - Identify local vs global variables
| - Refactor code to avoid global variables
| - Trace execution of programs with multiple scopes
| Teacher circulates, checks understanding of scope rules.

35-40 min | PLENARY + HOMEWORK

| Plenary: "Scope Detective"
| Display code with variables. Students hold up:
| - GREEN card if variable is LOCAL
| - RED card if variable is GLOBAL
| - YELLOW card if it will cause an ERROR
| Key question: "Your friend says 'Global variables make
| programming easier because all functions can see them.'
| What would you say?"
| Expected: "They cause bugs. It's hard to track changes.
| Better to pass data through parameters."
| Homework: Complete WORKSHEET 12A.
| "Find ONE example of global variable usage in code online
| (GitHub, tutorial). Explain why it could be a problem."

EAL Support:
- Use "house rooms" analogy throughout: global = living room (everyone can see), local = bedroom (only you can see your stuff)
- Visual diagram: boxes-within-boxes showing scope levels
- Colour-code: global variables (red border), local variables (blue border)
- Trace tables with scope annotations
- Bilingual: 局部变量 (jú bù biàn liàng = local variable) — 局部 means "partial/part" — only part of the program can see it

Differentiation:

Emerging:

- Focus on IDENTIFYING local vs global (not refactoring)
- Simple programs with only 1-2 functions
- Trace table with scope indicated
- Match-the-definition exercises
- True/false about scope rules

Developing:

- Standard worksheet — predict scope, identify errors

- Refactor simple programs to use parameters
- Explain why global variables are problematic
- Write functions with proper parameter passing

Secure:

- Extension: Refactor a complex 50-line program into scoped functions
- Explain variable lifetime concept
- Nested function scope (functions inside functions)
- Peer tutor Emerging students
- Challenge: "Write a program that demonstrates WHY global variables cause bugs — create a bug that only happens because of shared state"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- WORKSHEET 12A + 12B

- Scope visual diagram (nested boxes)

- Green/red/yellow cards for plenary

- Bilingual scope terminology card

LESSON 12.2 — Library Functions, Importing Modules & Creating Your Own Module

Topic: Library functions (math, random), importing modules, creating your own module, program structure with main program calling subroutines

Duration: 40 minutes

Lesson Type: Programming / Practical

Syllabus Ref: 2.2.5 (Library Routines), 2.2.5 (Modular Design)

Learning Objective:

By the end of this lesson, students will be able to import and use library functions from Python's math and random modules, create their own module file, and structure a program with a main program that calls subroutines.

Key Vocabulary:

- library / 库 (kù) — collection of pre-written functions

- module / 模块 (mó kuài) — Python file containing functions

- import / 导入 (dǎo rù) — bring a module into your program

- math module / 数学模块 — provides mathematical functions

- random module / 随机模块 — provides random number functions

- main program / 主程序 — top-level code that controls execution

- subroutine / 子程序 — function or procedure in a program

- modular design / 模块化设计 — breaking program into separate parts

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| "I want to use these functions in my program:
| - square root of 16
| - random number between 1 and 6 (like a dice)
| - pi value
| Python doesn't have these built-in. How do I get them?"
| Expected: "Import a module!"
| Show: import math, import random

3-12 min | DIRECT INSTRUCTION

| Write title: "Library Functions & Modular Design"
| CONCEPT 1: What is a library/module?
| "A module is a FILE full of functions that someone else wrote.
| You IMPORT it to use those functions in YOUR program."
| CONCEPT 2: The math module
| import math
| math.sqrt(16) → 4.0 (square root)
| math.pi → 3.14159... (constant)
| math.ceil(4.2) → 5 (round up)
| math.floor(4.9) → 4 (round down)
| math.pow(2, 3) → 8.0 (2 to the power 3)
| CONCEPT 3: The random module
| import random
| random.randint(1, 6) → random integer 1 to 6
| random.random() → random decimal 0.0 to 1.0
| random.choice(["A","B"]) → randomly picks one item
| random.shuffle(my_list) → shuffles list in place
| CULTURAL EXAMPLE:
| # Lucky draw for class prize
| students = ["Li Wei", "Wang Fang", "Zhang Ming",
| "Liu Hua", "Chen Jie"]
| winner = random.choice(students)
| print(f"The winner is: {winner}!")
| CONCEPT 4: Creating your own module
| Step 1: Create a file called mymath.py
| # mymath.py

| def add(a, b):
| return a + b
| def subtract(a, b):
| return a - b
| PI = 3.14159
| Step 2: In your main program:
| import mymath
| print(mymath.add(5, 3))
| print(mymath.PI)
| Step 3: Alternative import styles
| from mymath import add # Import just add()
| from mymath import * # Import everything
| import mymath as mm # Short alias
| print(mm.add(5, 3))
| CONCEPT 5: Program structure — main + subroutines
| Good structure:
| import statements
| CONSTANT definitions
| Function definitions
| def main():
| # Main program logic
| main()
| Why? Functions are defined BEFORE main() calls them.
| Also: if imported as a module, main() won't auto-run.

12-25 min | **GUIDED PRACTICE**

| Task 1 (teacher codes live):
| Dice rolling game using random module.
| import random
| def roll_dice():
| return random.randint(1, 6)
| def roll_two_dice():
| return roll_dice() + roll_dice()
| print(roll_two_dice())
| Task 2 (pairs):
| "Write a function that uses math.sqrt() to check if a triangle
| with sides a, b, c is a right-angled triangle

| $(a^2 + b^2 = c^2)$."

| Answer:

| import math

| def is_right_angled(a, b, c):

| sides = sorted([a, b, c])

| return math.isclose(sides[0]**2 + sides[1]**2, sides[2]**2)

| Task 3 (teacher codes):

| Create a simple mytools.py module with useful functions.

| Then import and use it.

25-35 min | INDEPENDENT PRACTICE

| Distribute WORKSHEET 12B.

| Students:

| - Use math and random functions in programs

| - Create their own utility module

| - Build a properly structured program with main() function

| - Combine skills from all previous weeks

| Teacher circulates, helps with import syntax.

35-40 min | PLENARY + HOMEWORK

| Plenary: "Module Match"

| Match the function to the module:

| - sqrt() → math

| - randint() → random

| - choice() → random

| - pi → math

| - shuffle() → random

| Key question: "Why would you create your own module instead

| of putting all code in one file?"

| Answer: "Reuse functions across programs. Teamwork — different

| people work on different modules. Organise code logically."

| Homework: Complete WORKSHEET 12B.

| "Create your own module called utilities.py with at least

| 3 useful functions (e.g., get_positive_number(),

| print_header(), is_valid_email()). Then write a main

| program that imports and uses them."

EAL Support:

- "Module" = 模块 (mó kuài = building block) — like LEGO bricks

- Import = bringing tools into your toolbox

- Use visual: draw toolbox, add tools (functions) from different modules

- Program structure: use building analogy (foundation=imports, walls=functions, roof=main)

- Bilingual: math=数学, random=随机, import=导入, module=模块

Differentiation:

Emerging:

- Use provided import statements (don't need to remember syntax)
- Focus on math.sqrt() and random.randint() only
- Pre-written module; student only imports and uses
- Simple programs with single import

Developing:

- Standard worksheet — use math and random functions
- Create a simple module with 2-3 functions
- Structure program with main()
- Import using different styles (import x, from x import y)

Secure:

- Extension: Create a complete game using math and random
- Multiple custom modules with related functions
- Use less common math functions (factorial, trig)
- Peer tutor Emerging students
- Challenge: "Write a number guessing game that uses random for the secret, math for hint calculations (e.g., 'the number is prime'), and your own module for input validation"

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Mini-whiteboards and markers

- WORKSHEET 12B

- Module import syntax reference card

- Python standard library reference

- Bilingual module terminology card

WORKSHEET 12A — Scope: Local vs Global Variables

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Vocabulary (词汇)

178. The area where a variable can be used is called its _____. (作用域)
179. A variable created inside a function is called a _____ variable. (局部)

180. A variable created outside all functions is called a _____ variable. (全局)

181. Sending data into a function through its definition is called _____ passing. (参数)

182. An unexpected change to data caused by a function is called a _____. (副作用)

Section B: Local or Global? (局部还是全局?)

For each variable in this program, write L (Local) or G (Global).

```
total = 0                # total is: _____
def calculate():
    score = 85           # score is: _____
    bonus = 10           # bonus is: _____
    result = score + bonus   # result is: _____
    return result
def display():
    print(total)         # total here is: _____
    message = "Done"     # message is: _____
    print(message)
total = calculate()      # total is: _____
display()
```

183. How many different local variables are there? _____

7. How many global variables are there? _____

Section C: Predict the Output (预测输出)

8. What will this program print?

```
def func_a():
    x = 10
    print(f"A: {x}")
def func_b():
    x = 20
    print(f"B: {x}")
x = 5
print(f"Main: {x}")
func_a()
func_b()
```

print(f"Main: {x}")

Output:

9. What will this program print?

```
def double(x):
    return x * 2
def triple(x):
    return x * 3
x = 4
print(double(x))
print(triple(x))
print(x)
```

Output:

Section D: Fix the Code (修复代码)

184. This code has a scope problem. Fix it by using parameters and return.

```
# BAD VERSION (using global variables incorrectly)
total = 0
def add_score():
    global total        # BAD!
    score = int(input("Enter score: "))
    total = total + score
add_score()
add_score()
print(total)
# YOUR FIXED VERSION (no global keyword!):
```

Section E: Refactoring Task (重构任务)

185. This monolithic program uses no functions. Refactor it into properly-scoped functions with parameters.

```
# BAD: everything in one block
print("Student Mark Calculator")
name = input("Enter name: ")
mark1 = int(input("Mark 1: "))
mark2 = int(input("Mark 2: "))
mark3 = int(input("Mark 3: "))
total = mark1 + mark2 + mark3
average = total / 3
```

print(f"{name}: Total={total}, Average={average:.1f}")

YOUR REFACTORED VERSION:

WORKSHEET 12B — Library Functions, Modules & Program Structure

Name: _____ Class: _____ Date: _____

姓名: _____ 班级: _____ 日期: _____

Section A: Match the Function to the Module (匹配函数与模块)

Write math or random for each function:

186. sqrt() → _____
187. randint() → _____
188. pi → _____
189. random() → _____
190. ceil() → _____
191. choice() → _____
192. floor() → _____
193. shuffle() → _____

Section B: Import Styles (导入方式)

9. Write the import statement to:

a) Import the entire math module:

b) Import just sqrt from math:

c) Import random and give it the alias rnd:

d) Import everything from random:

Section C: Using Library Functions (使用库函数)

10. Write a program that:

a) Asks the user for the radius of a circle

b) Uses math.pi to calculate the area (pi × r^2)

c) Uses math.pi to calculate the circumference (2 × pi × r)

d) Prints both results rounded to 2 decimal places

11. Write a program that simulates rolling a dice 10 times:

a) Use random.randint(1, 6) for each roll

b) Count how many times each number (1-6) appears

c) Print the results

Section D: Create Your Own Module (创建自己的模块)

194. Imagine you create a file called utils.py with these functions:

utils.py

```
def get_positive_number(prompt):
    while True:
        value = float(input(prompt))
        if value > 0:
            return value
        print("Please enter a positive number.")
def print_header(title):
    print("=" * 30)
    print(title.center(30))
    print("=" * 30)
```

Write the main program that imports and uses these functions:

Section E: Final Challenge — Complete Program (最终挑战)

13. DICE GAME:

Write a complete, well-structured program that:

a) Imports random and math

b) Defines these functions:

- roll_dice() — returns random 1-6
- play_round() — rolls 2 dice, returns the sum
- play_game(n) — plays n rounds, returns total score

c) Has a main() function that:

- Asks how many rounds to play
- Calls play_game()
- Calculates and prints the average score per round
- Announces if the player had "good luck" (average > 7) or "bad luck" (average ≤ 7)

[Write your code in Thonny. Save as dice_game.py]

Structure hint:

import statements

function definitions

```
def main():
    # your code
main()
```

ANSWER KEY — WEEK 12 WORKSHEETS

WORKSHEET 12A ANSWERS:

Section A:

195. scope / 作用域

196. local / 局部
197. global / 全局
198. parameter / 参数
199. side effect / 副作用

Section B:

```
total = 0                # total is: G
def calculate():
    score = 85           # score is: L
    bonus = 10           # bonus is: L
    result = score + bonus   # result is: L
    return result
def display():
    print(total)         # total here is: G
    message = "Done"     # message is: L
    print(message)
```

200. 4 local variables (score, bonus, result, message)
201. 1 global variable (total)

Section C:

8. Output:

Main: 5

A: 10

B: 20

Main: 5

Explanation: Each function has its own x. The global x (5) is never changed by the functions. They use their own local x values.

9. Output:

8 (double(4) = 8)

12 (triple(4) = 12)

4 (the original x is unchanged)

Section D:

10. Fixed version:

```
def add_score(current_total):
    score = int(input("Enter score: "))
    return current_total + score
```

```
total = 0
total = add_score(total)
total = add_score(total)
print(total)
```

Section E:

11. Refactored version:

```
def get_student_info():
    name = input("Enter name: ")
    return name
def get_marks():
    marks = []
    for i in range(3):
        mark = int(input(f"Mark {i+1}: "))
        marks.append(mark)
    return marks
def calculate_total(marks):
    return sum(marks)
def calculate_average(marks):
    return sum(marks) / len(marks)
def display_results(name, marks):
    total = calculate_total(marks)
    average = calculate_average(marks)
    print(f"{name}: Total={total}, Average={average:.1f}")
def main():
    print("Student Mark Calculator")
    name = get_student_info()
    marks = get_marks()
    display_results(name, marks)
main()
```

(Alternative acceptable versions with similar structure)

WORKSHEET 12B ANSWERS:

Section A:

202. math
203. random

204. math
205. random
206. math
207. random
208. math
209. random

Section B:

210. a) import math

b) from math import sqrt

c) import random as rnd

d) from random import *

Section C:

10. Sample:

```
import math
radius = float(input("Enter radius: "))
area = math.pi * radius * radius
circumference = 2 * math.pi * radius
print(f"Area: {area:.2f}")
print(f"Circumference: {circumference:.2f}")
```

11. Sample:

```
import random
counts = [0, 0, 0, 0, 0, 0]  # counts for 1,2,3,4,5,6
for _ in range(10):
    roll = random.randint(1, 6)
    counts[roll - 1] = counts[roll - 1] + 1
for i in range(6):
    print(f"{i+1}: {counts[i]} times")
```

Section D:

12. Sample main program:

```
import utils
def main():
    utils.print_header("Area Calculator")
    width = utils.get_positive_number("Enter width: ")
    height = utils.get_positive_number("Enter height: ")
```

```
    area = width * height
    print(f"Area: {area}")
main()
```

Or with specific import:

```
from utils import get_positive_number, print_header
def main():
    print_header("Area Calculator")
    width = get_positive_number("Enter width: ")
    height = get_positive_number("Enter height: ")
    area = width * height
    print(f"Area: {area}")
main()
```

Section E:

13. Complete dice game:

```
import random
import math
def roll_dice():
    return random.randint(1, 6)
def play_round():
    die1 = roll_dice()
    die2 = roll_dice()
    total = die1 + die2
    print(f"Rolled: {die1} + {die2} = {total}")
    return total
def play_game(n):
    total_score = 0
    for i in range(n):
        round_score = play_round()
        total_score = total_score + round_score
    return total_score
def main():
    rounds = int(input("How many rounds? "))
    total = play_game(rounds)
    average = total / rounds
    print(f"\nTotal score: {total}")
```

```
    print(f"Rounds played: {rounds}")
    print(f"Average per round: {average:.2f}")
    if average > 7:
        print("Good luck!")
    else:
        print("Bad luck! Try again.")
main()
```

CIE-Style Exam Question (Bonus, 6 marks):

"A modular program is being designed to process student examination marks.
The program will use separate modules for input, calculation, and output.

(a) State TWO benefits of using a modular design approach. [2 marks]

(b) Explain why global variables should be avoided in modular programs. [2 marks]

(c) A function called CalculateGrade takes a mark as a parameter and returns a grade (A, B, C, D, or F). Write the function header in pseudocode. [2 marks]"

Mark Scheme:

(a) Any TWO from:

- Easier to test individual modules [1]
- Code reuse in other programs [1]
- Multiple programmers can work simultaneously [1]
- Easier to maintain/debug [1]
- Better organisation/readability [1]

(b) Any TWO from:

- Global variables can be changed by any module [1]
- Makes it difficult to track where changes occur [1]
- Creates dependencies between modules [1]
- Functions become less reusable [1]
- Harder to test modules independently [1]

(c)

- FUNCTION CalculateGrade(Mark) [1]
- RETURN Grade (or appropriate return statement) [1]

END OF WEEKS 7-12

SUMMARY OF TOPICS COVERED:

Week 7: Iteration — FOR loops (range, list iteration), WHILE loops (condition-controlled, sentinel values, infinite loops)

Week 8: 1D Arrays/Lists — creating, indexing, operations (append, insert, remove, pop, len), traversing with FOR, max/min/sum/average, linear search

Week 9: 2D Arrays/Nested Lists — rows and columns, nested lists, double indexing [row][col], nested FOR loops, row/column sums

Week 10: String Handling — slicing, methods (upper, lower, find, replace, split), ASCII (ord, chr), reversing, palindromes, text analysis

Week 11: Functions — defining (def), parameters/arguments, return values, built-in vs user-defined, procedures, nested calls, calculator

Week 12: Scope & Modular Programming — local vs global, scope rules, library functions (math, random), importing modules, creating modules, program structure

TOTAL DELIVERABLES:

12 detailed lesson plans (2 per week x 6 weeks)

12 worksheets (XA + XB per week)

6 comprehensive answer keys

Bilingual vocabulary throughout (English + Chinese + pinyin)

Differentiation for Emerging / Developing / Secure

CIE-style exam questions with mark schemes

Python code for Thonny IDE